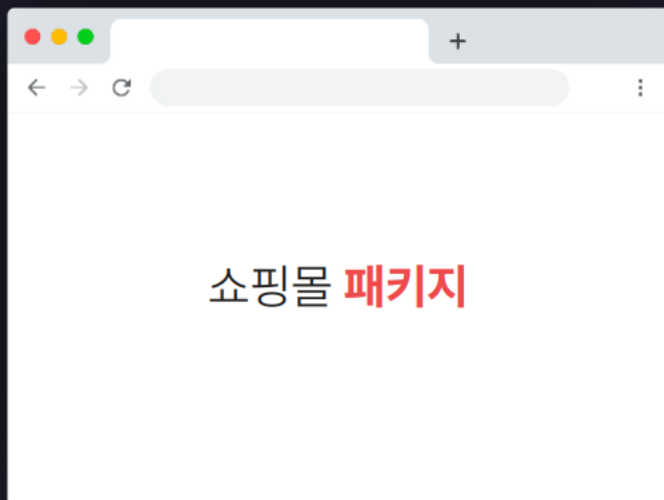


React 강의

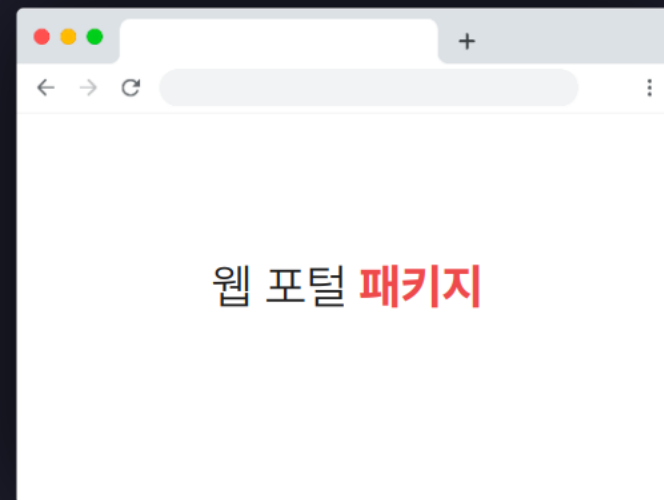
프로젝트(Project)

패키지

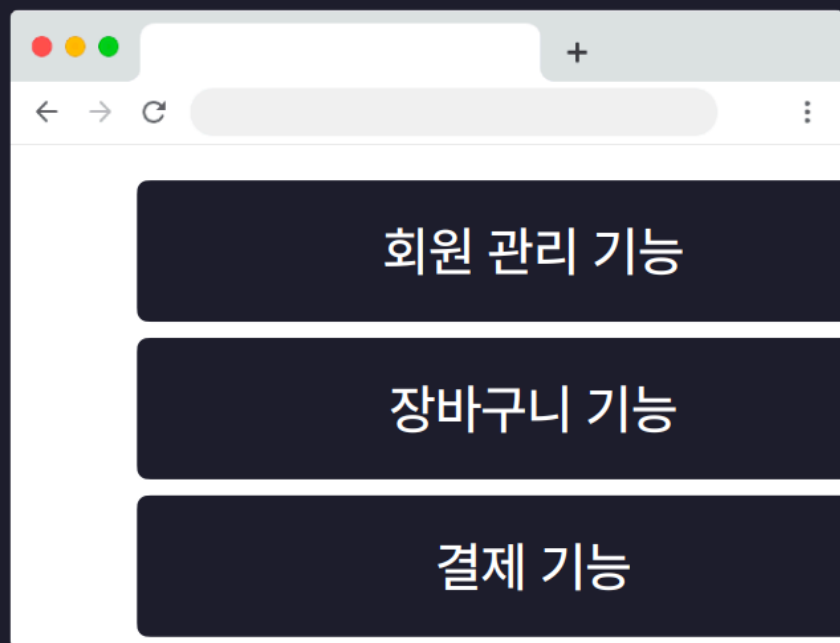
Node.js에서 사용하는 프로그램의 단위



@한 입 크기로 잘라먹는 리액트



모듈(Module)이란?



```
import React, { useEffect, useReducer, useRef } from "react";
import "./App.css";
import { BrowserRouter, Route, Routes } from "react-router-dom";

import Home from "./pages/Home";
import New from "./pages/New";
import Edit from "./pages/Edit";
import Diary from "./pages/Diary";

const reducer = (state, action) => {
  let newState = [];
  switch (action.type) {
    case "INIT": {
      return action.data;
    }
    case "CREATE": {
      newState = [action.data, ...state];
      break;
    }
    case "REMOVE": {
      newState = state.filter((it) => it.id !== action.targetId);
      break;
    }
    case "EDIT": {
      newState = state.map((it) =>
        it.id === action.data.id ? { ...action.data } : it
      );
      break;
    }
    default:
      return state;
  }
};

localStorage.setItem("diary", JSON.stringify(newState));
return newState;
};

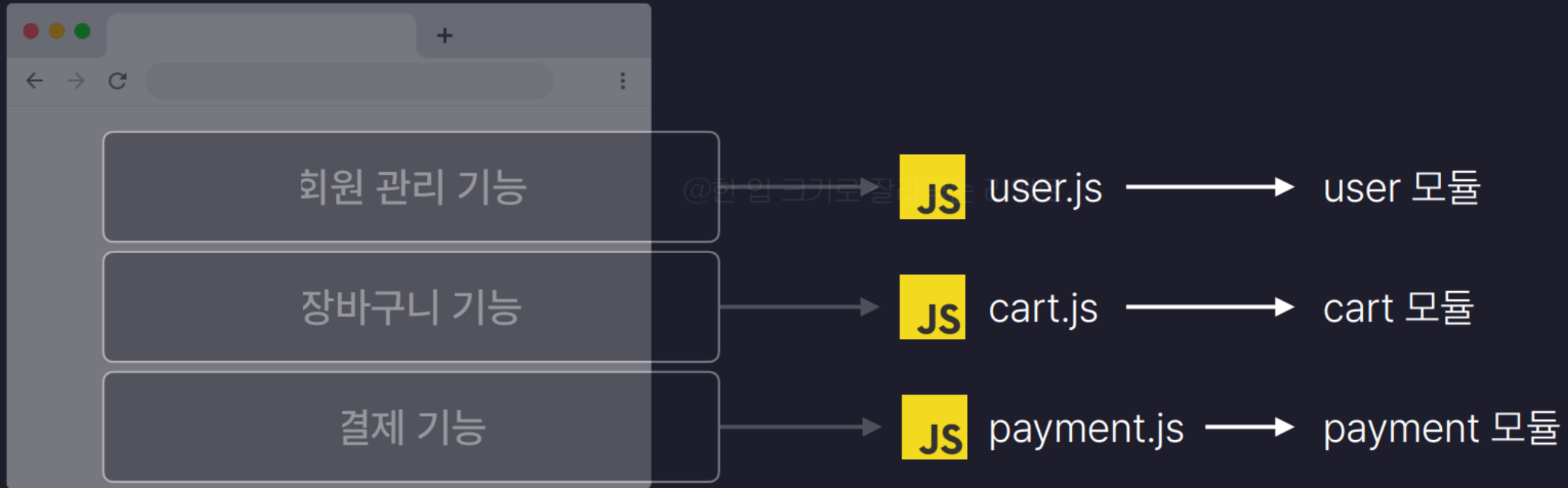
export const DiaryStateContext = React.createContext();
export const DiaryDispatchContext = React.createContext();

function App() {
  const [data, dispatch] = useReducer(reducer, []);

  useEffect(() => {
    const localData = localStorage.getItem("diary");
    if (localData) {
      const diaryList = JSON.parse(localData).sort(
        (a, b) => parseInt(b.id) - parseInt(a.id)
      );

      if (diaryList.length >= 1) {
        dataId.current = parseInt(diaryList[0].id) + 1;
        dispatch({ type: "INIT", data: diaryList });
      }
    }
  }, []);
```

모듈(Module)이란?



모듈 시스템(Module System)이란?

모듈 시스템

모듈을 생성하고, 불러오고, 사용하는 등의
모듈을 다루는 다양한 기능을 제공하는 시스템

@한 입 크기로 잘라먹는 리액트

JS

user 모듈

JS

cart 모듈

JS

payment 모듈

모듈 시스템(Module System)이란?

JavaScript의 모듈 시스템

Common JS (CJS)

@학업 크기로 정리하는 리액트
ES Module (ESM)

AMD

UMD

...

React의 기술적인 특징



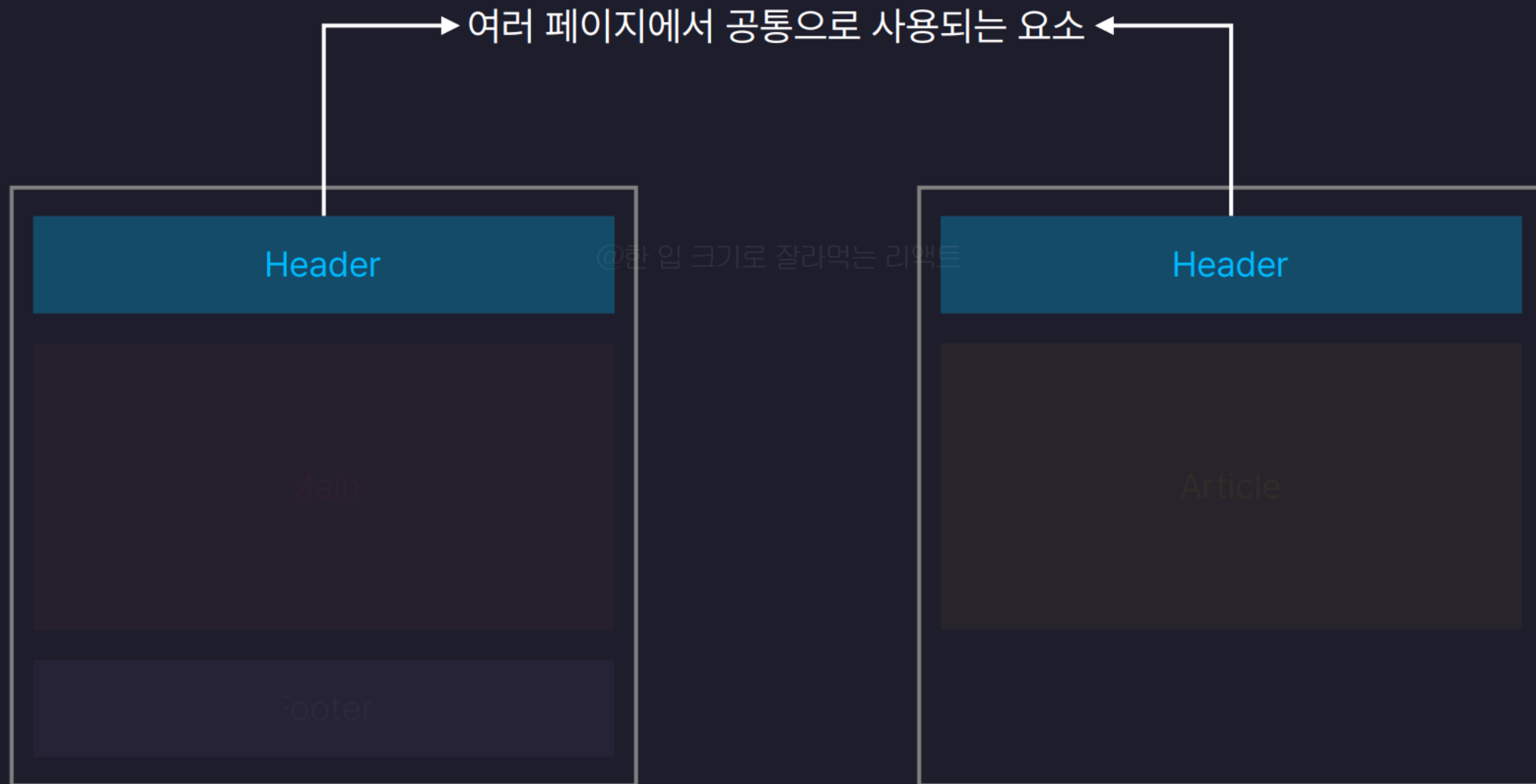
@한 입 크기로 잘라먹는 리액트

1. 컴포넌트를 기반으로 UI를 표현한다

2. 화면 업데이트 구현이 쉽다

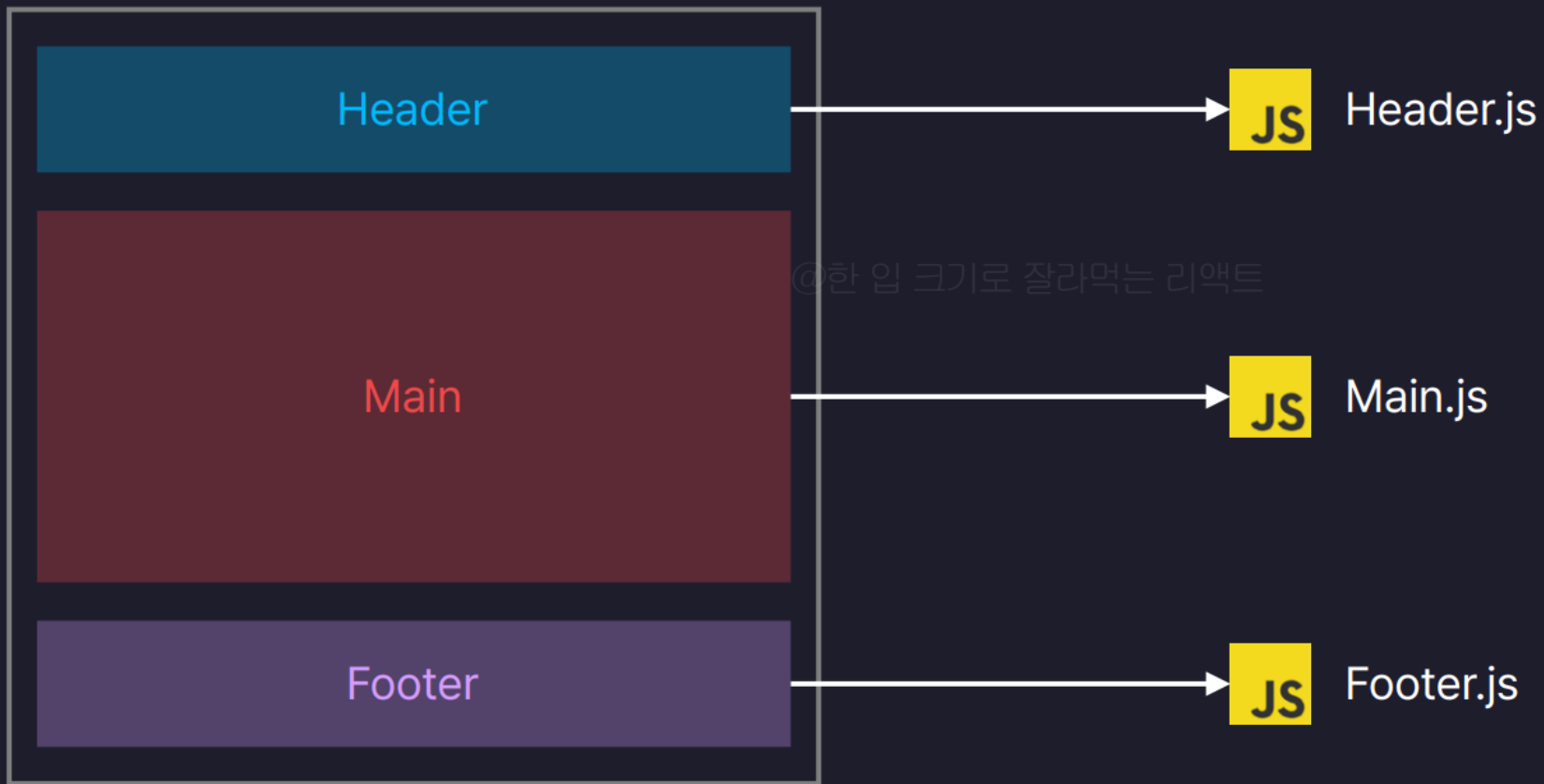
3. 화면 업데이트가 빠르게 처리된다

React의 기술적인 특징 1. 컴포넌트를 기반으로 UI를 표현한다

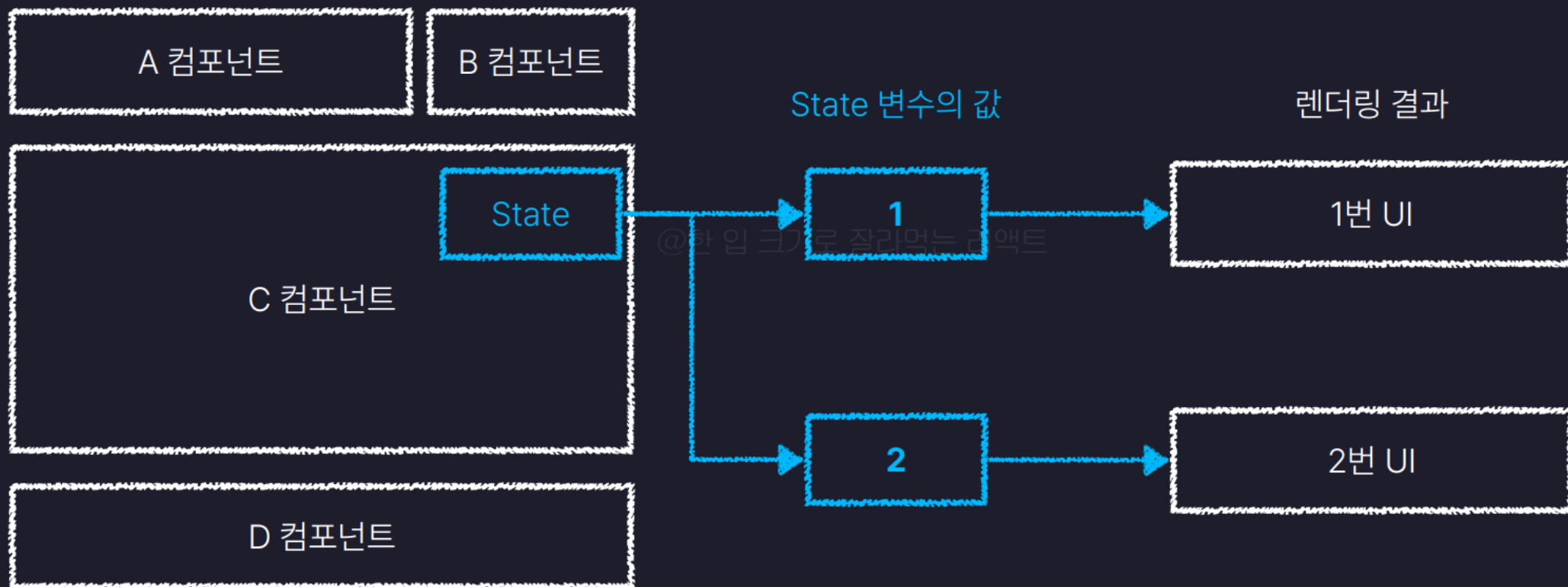


React의 기술적인 특징 1. 컴포넌트를 기반으로 UI를 표현한다

< 웹 페이지 >

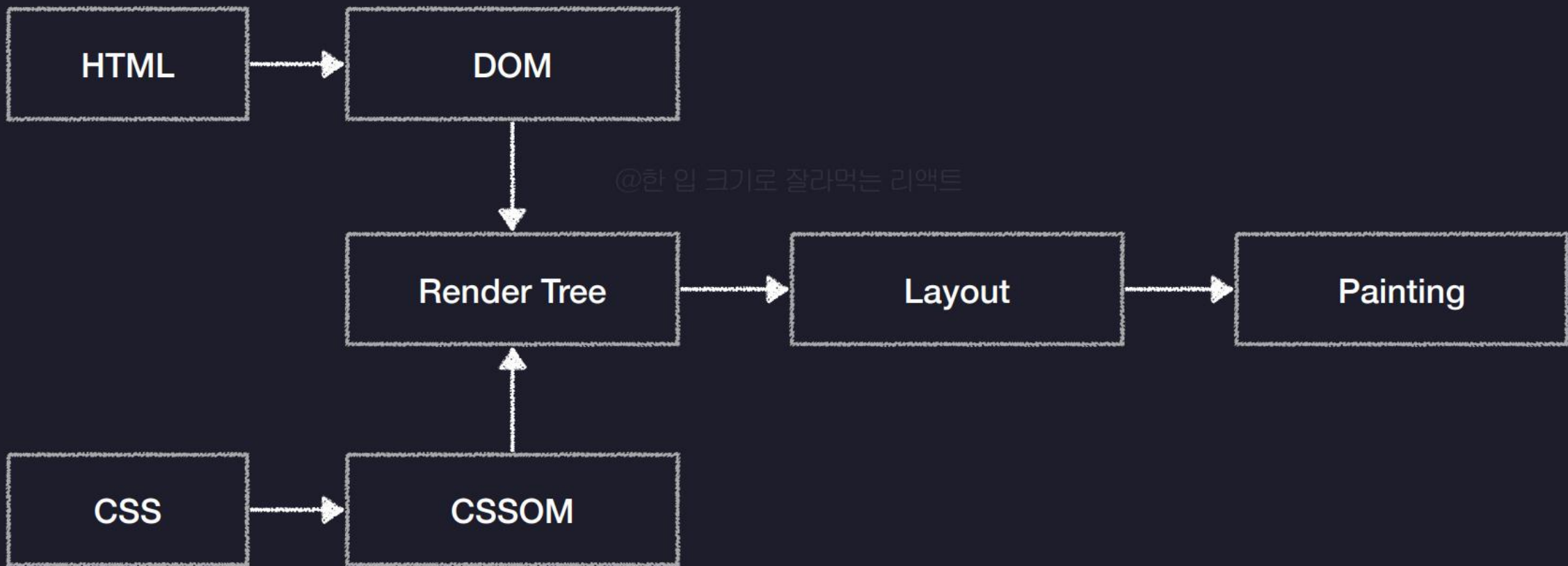


React의 기술적인 특징 2. 화면 업데이트 구현이 쉽다



React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

브라우저의 렌더링 과정 (Critical Rendering Path)

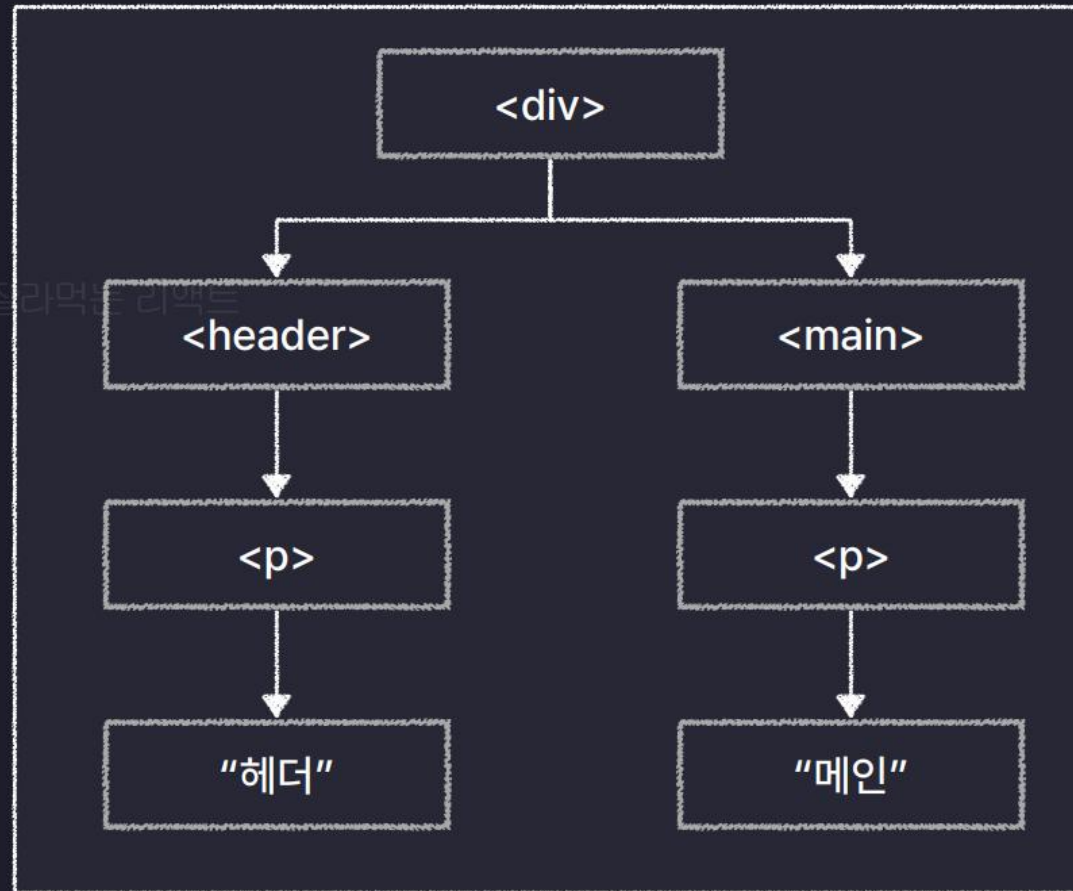


React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

HTML

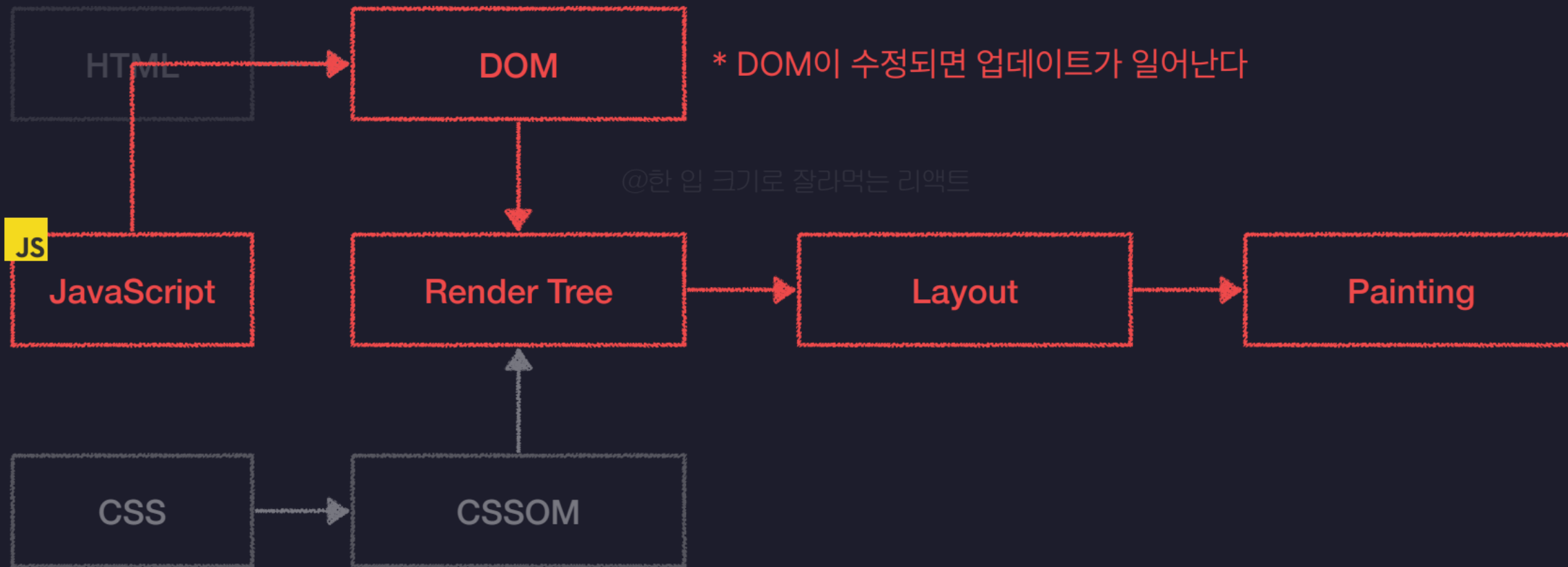


DOM (Document Object Model)



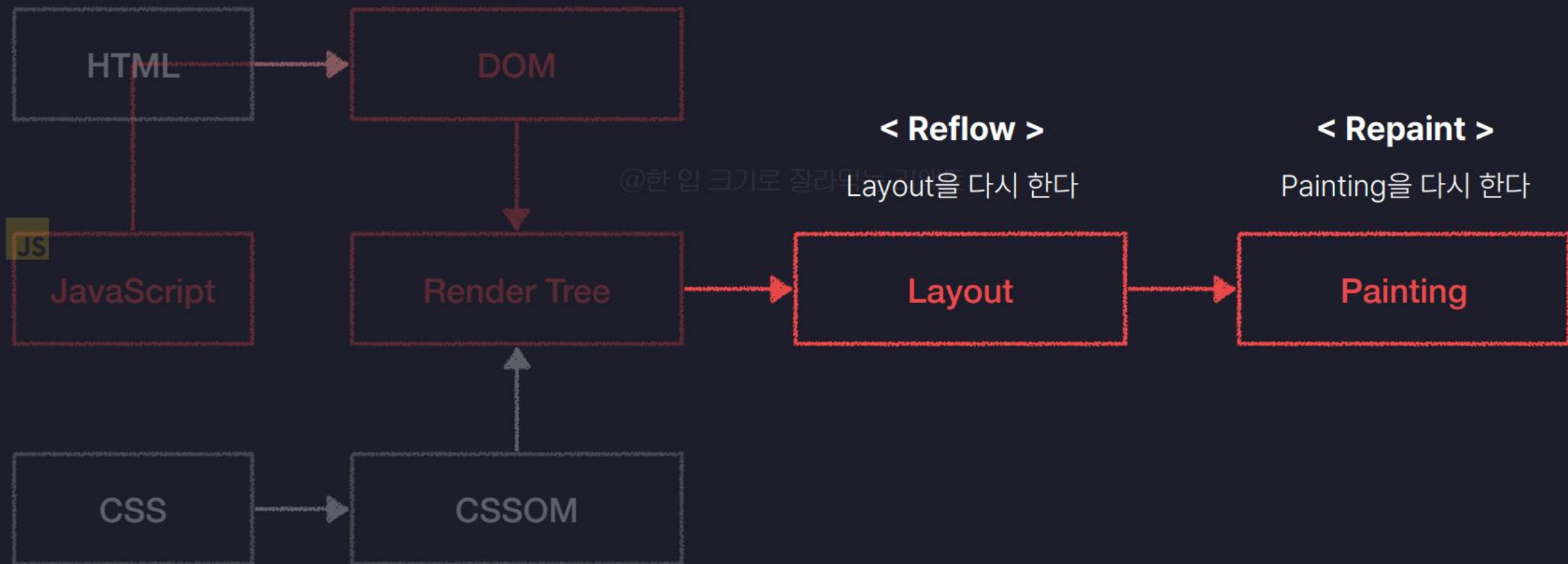
React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

JavaScript가 DOM을 수정하면 업데이트가 발생한다



React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

주의! Layout, Painting은 매우 오래걸리는 과정임



React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

index.html

```
11  <script>
12    function onClick() {
13      const $ul = document.getElementById("ul");
14      for (let i = 0; i < 3000; i++) {
15        $ul.innerHTML += `<li>${i}</li>`;
16      }
17    }
18  </script>
19  <body>
20    <button onclick="onClick()">리스트 추가하기</button>
21    <ul id="ul"></ul>
22  </body>
```

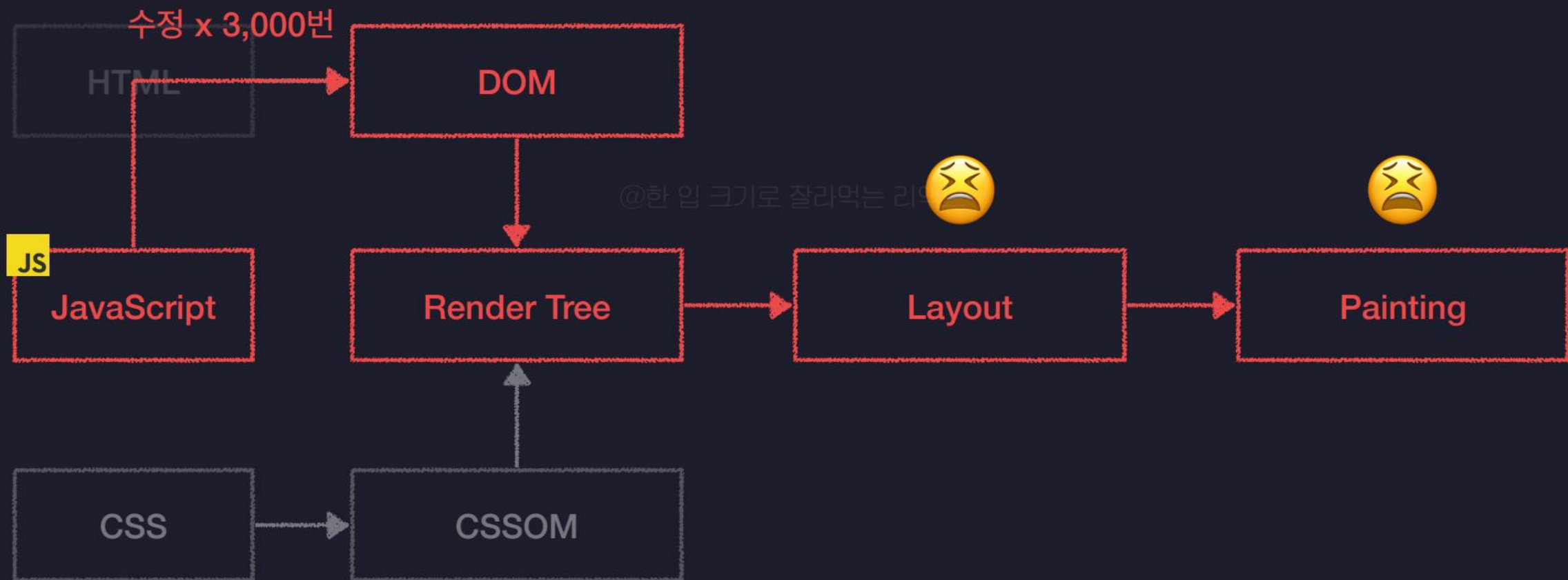
렌더링 결과

리스트 추가하기!

- 0
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

JavaScript가 DOM을 수정하면 업데이트가 발생한다



React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

index.html

```
11 <script>
12   function onClick() {
13     const $ul = document.getElementById("ul");
14     for (let i = 0; i < 3000; i++) {
15       $ul.innerHTML += `<li>${i}</li>`;
16     }
17   }
18 </script>
19 <body>
20   <button onclick="onClick()">리스트 추가하기</button>
21   <ul id="ul"></ul>
22 </body>
```

🤨 Super Bad Practice

- 3,000번 DOM을 수정
- 성능 측정 결과 : **4,500ms**

React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

```
11 <script>
12   function onClick() {
13     const $ul = document.getElementById("ul");
14     let list = "";
15
16     for (let i = 0; i < 3000; i++) {
17       list += `<li>${i}</li>`;
18     }
19
20     $ul.innerHTML = list;
21   }
22 </script>
23 <body>
24   <button onclick="onClick()">리스트 추가하기</button>
25   <ul id="ul"></ul>
26 </body>
```

✨ Good Practice

- DOM은 딱 한번만 수정함
- 성능 측정 결과 : **250ms**
(약 22배 개선 됨)

React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

React는 이 과정을 자동으로 해줌
(Feat. Virtual DOM)

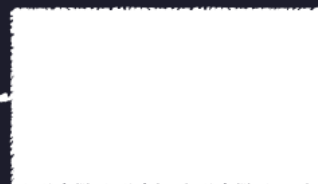


@한 입 크기로 씹어먹는 리액트

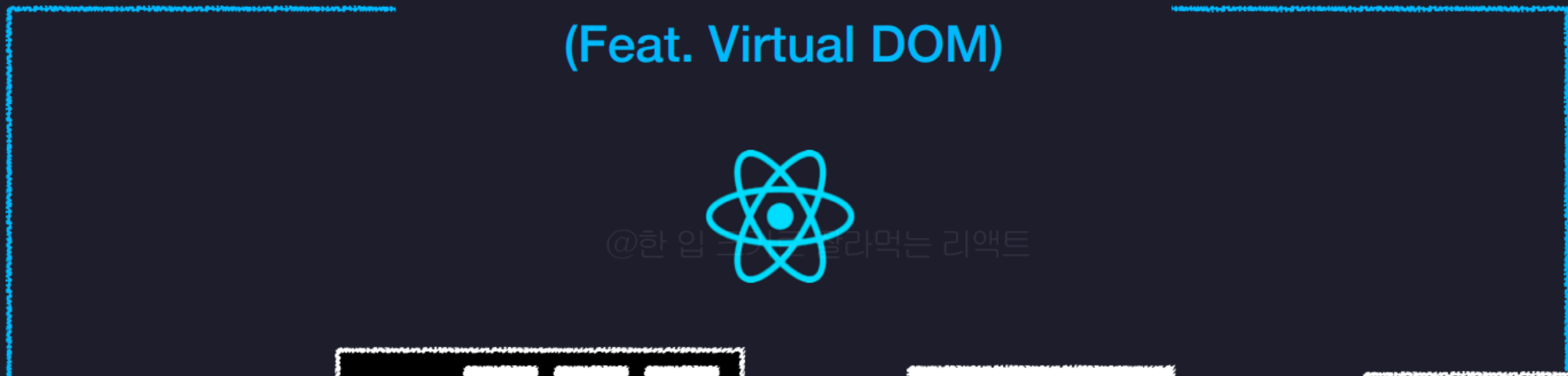
다양한 업데이트



동시에 발생한
업데이트를 모음



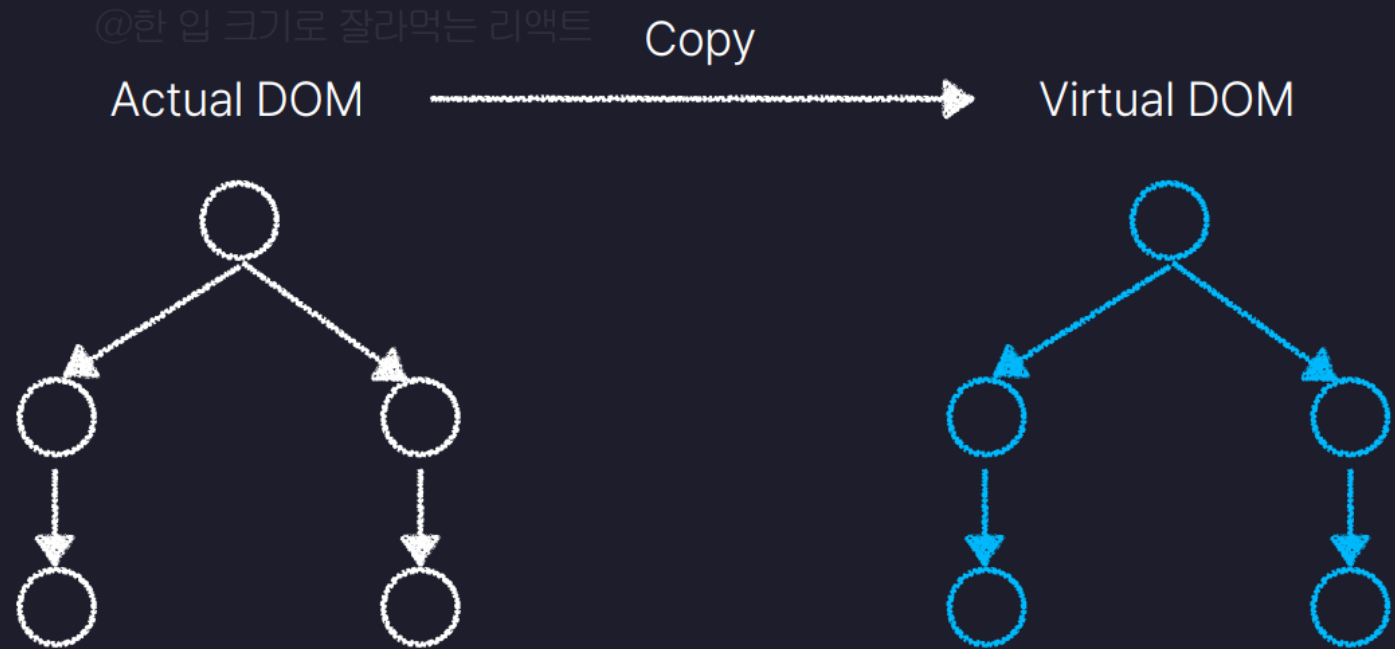
다 모였다면
한번에 수정



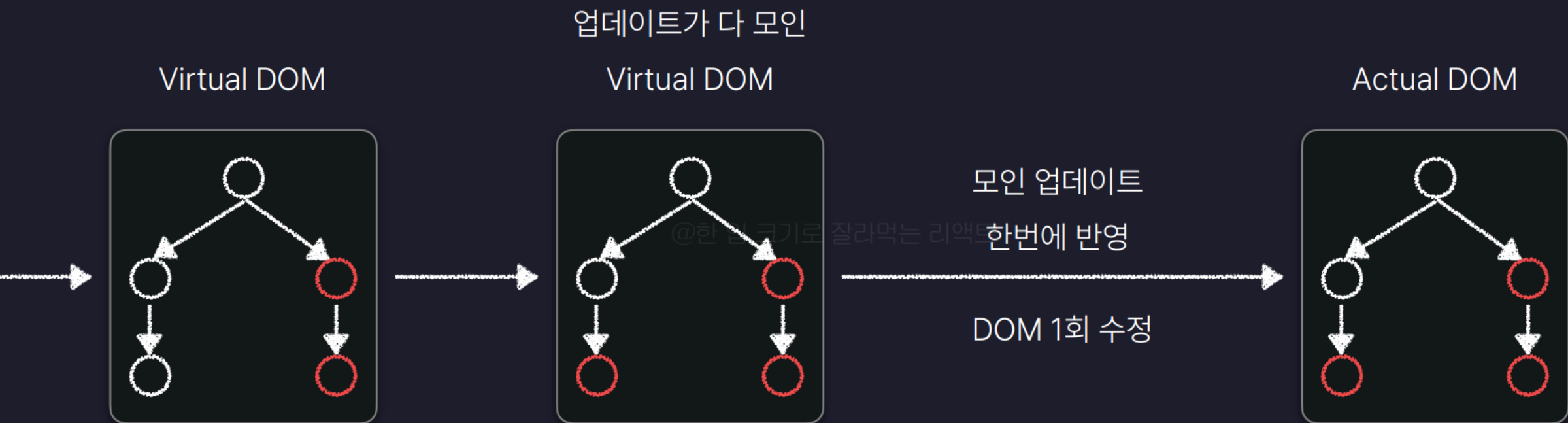
React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다

Virtual DOM이란?

- DOM을 자바스크립트 객체로 흉내낸 것으로 일종의 복제판이라고 생각하면 된다
- React는 업데이트가 발생하면 실제 DOM을 수정하기 전에 이 가상의 복제판 DOM에 먼저 반영해본다.
 - 연습 스윙 같은 느낌이라고 생각하면 된다.



React의 기술적인 특징 3. 화면 업데이트가 빠르게 처리된다



React App 생성하기



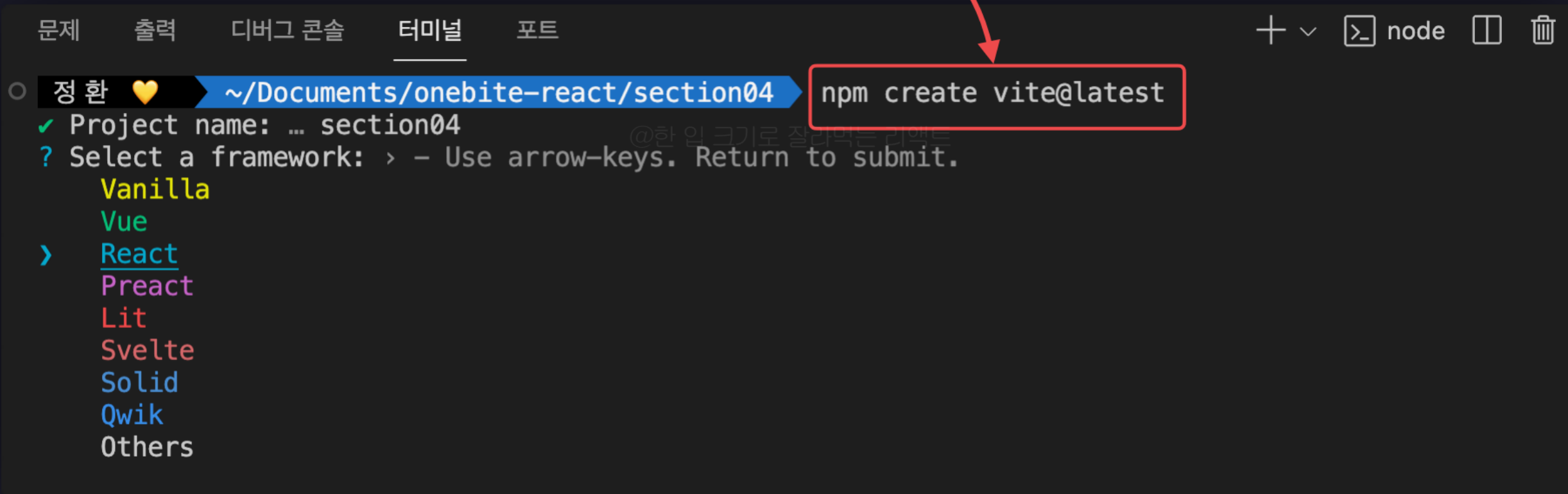
@한 입 크기로 씹어먹는 리액트

Vite

차세대 프론트엔드 개발 툴
기본 설정이 적용된 React App 생성 가능

React App은 어떻게 구동되는걸까?

1. React App 생성(with Vite)



The screenshot shows a VS Code terminal window with the following content:

```
문제  출력  디버그 콘솔  터미널  포트
```

○ 정 환 ♥ ~/Documents/onebite-react/section04 npm create vite@latest

✓ Project name: ... section04

? Select a framework: > - Use arrow-keys. Return to submit.

- Vanilla
- Vue
- > React
- Preact
- Lit
- Svelte
- Solid
- Qwik
- Others

A red arrow points from the text "1. React App 생성(with Vite)" to the command "npm create vite@latest" in the terminal.

React App은 어떻게 구동되는걸까?

2. React App 가동



The screenshot shows a VS Code terminal window with the following content:

```
문제  출력  디버그 콘솔  터미널  포트
○ 정 환 ♥ ~/Documents/onebite-react/section04 npm run dev
> section04@0.0.0 dev
> vite

VITE v5.0.12 ready in 378 ms

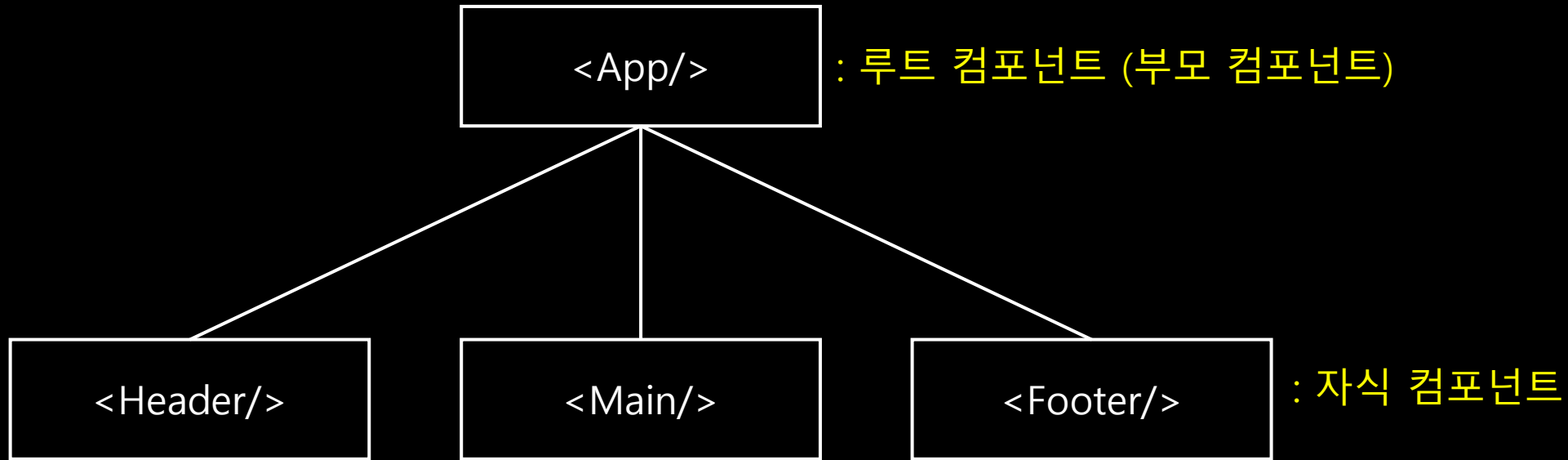
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

A red arrow points from the text "2. React App 가동" to the `npm run dev` command in the terminal. The terminal output shows the Vite development server starting successfully on `http://localhost:5173/`.

React Application?



컴포넌트 트리



JSX = JavaScript Extensions : 확장된 자바스크립트의 문법

React Component : JSX로 된 HTML을 리턴하는 함수를 컴퍼넌트라함, 첫글자는 꼭! 대문자

```
function Footer() {  
  return (  
    <footer>  
      <h1>footer</h1>  
    </footer>  
  );  
}
```

← JavaScript

@한 입 크기로 잘라먹는 리액트
← HTML

JSX의 중괄호에는 "JavaScript 표현식"만 넣을 수 있다

React Component

```
function Footer() {  
  const myName = "홍길동";  
  
  return (  
    <footer>  
      <h1>안녕 내 이름은 {myName}이야</h1>  
      <h1>footer</h1>  
    </footer>  
  );  
}
```

JavaScript 표현식

- 값 : 숫자, 문자열, 배열 (ex. {10} , ...)
- 변수이름
- 연산식 (삼항연산식까지 가능)

True, undefined, null , object 자체
(에러는 아니지만 표현은 안됨! 굳이 하고싶으면 { String(true) }
으로 명시적 캐스팅을

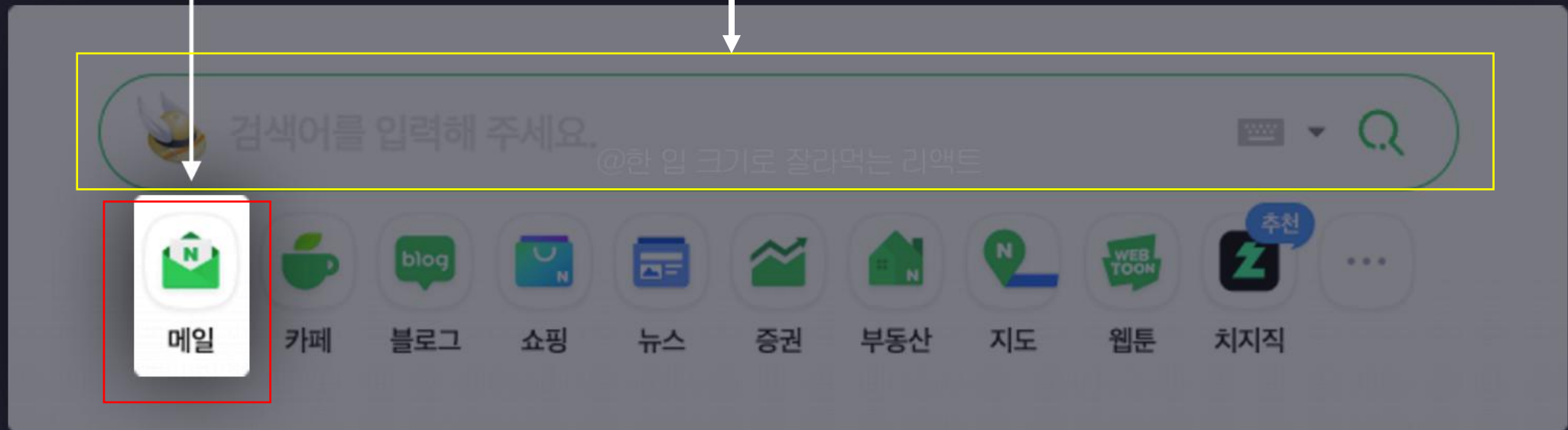
Jsx 주의사항!

1. 중괄호 내부에는 자바스크립트 표현식만 넣을 수 있다.
2. 숫자, 문자열, 배열 값만 렌더링 된다. (위 참조)
3. 모든 태그는 닫혀있어야 한다.
4. 최상위 태그는 반드시 하나여야만 한다.
(빈태그 <></>도 가능)

Props란? : Properties의 줄임말로 자식 컴포넌트에 전달되는 값

Button 컴포넌트

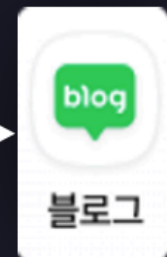
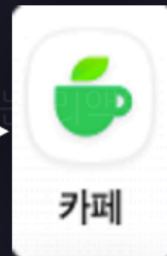
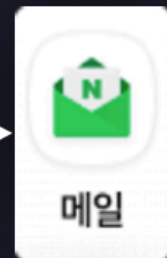
Search 컴포넌트



Button 컴포넌트

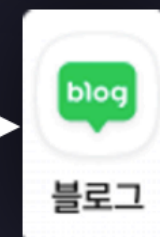
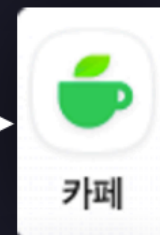
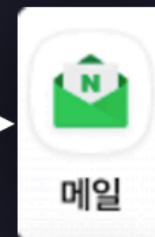


```
function App() {  
  return (  
    <>  
    <Button/>  
    <Button/>  
    <Button/>  
    ...  
  </>  
);  
}
```



Props란? : Properties의 줄임말로 자식 컴포넌트에 전달되는 값

```
function App() {  
  return (  
    <>  
      <Button text={"메일"} img={"mail.png"} />  
      <Button text={"카페"} img={"cafe.png"} />  
      <Button text={"블로그"} img={"blog.png"} />  
      ...  
    </>  
  );  
}
```



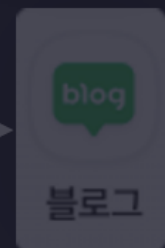
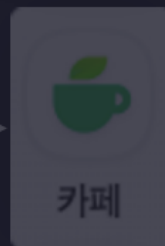
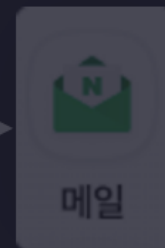
Props란?

```
function App() {  
  return (  
    <>  
    <Button text={"메일"} img={"mail.png"} />  
    <Button text={"카페"} img={"cafe.png"} />  
    <Button text={"블로그"} img={"blog.png"} />  
    ...  
  </>  
);  
}
```

↓

값을 전달할 수 있음

Props



jsx

```
function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

// 사용

```
<Greeting name="Alice" />
```

jsx

```
function Double({ number }) {  
  return <p>Double: {number * 2}</p>;  
}
```

// 사용

```
<Double number={5} />
```

jsx

```
function Wrapper({ children }) {  
  return <div className="box">{children}</div>;  
}
```

// 사용

```
<Wrapper>  
  <p>This is inside the wrapper.</p>  
</Wrapper>
```

jsx

```
function Status({ isOnline }) {  
  return <p>{isOnline ? 'Online' : 'Offline'}</p>;  
}
```

// 사용

```
<Status isOnline={true} />
```

jsx

```
function List({ items }) {  
  return (  
    <ul>  
      {items.map((item, idx) => <li key={idx}>{item}</li>)}  
    </ul>  
  );  
}
```

// 사용

```
<List items={['Apple', 'Banana', 'Cherry']} />
```

jsx

```
function ColoredText({ color, text }) {  
  return <p style={{ color }}>{text}</p>;  
}
```

// 사용

```
<ColoredText color="red" text="This is red!" />
```

jsx

```
function Button({ onClick }) {  
  return <button onClick={onClick}>Click me</button>;  
}
```

// 사용

```
<Button onClick={() => alert('Clicked!')} />
```

jsx

```
function Avatar({ src, alt }) {  
  return <img src={src} alt={alt} width="100" />;  
}
```

// 사용

```
<Avatar src="https://via.placeholder.com/100" alt="Profile" />
```

jsx

```
function LinkButton({ url, label }) {  
  return <a href={url} target="_blank" rel="noreferrer">{label}</a>;  
}
```

// 사용

```
<LinkButton url="https://react.dev" label="React Docs" />
```

이벤트 핸들링이란?

Event Handling

@한 입 크기로 잘라먹는 리액트

웹 내부에서 발생하는 사용자의 행동

ex) 버튼 클릭, 메시지 입력, 스크롤 등등

Event Handling

@한 입 크기로 잘라먹는 리액트

다루다, 취급하다, 처리하다

Event Handling

@한 입 크기로 잘라먹는 리액트

이벤트가 발생했을 때 그것을 처리하는 것

ex) 버튼 클릭시 경고창 노출

React Hooks란?



Class 컴포넌트

모든 기능을 이용할 수 있음
ex) State, Ref, etc...

React Hooks



Function 컴포넌트

기 렌더링만 할 수 있음

리액트 클래스형 컴포넌트에서 이용하던 코드를 함수형 컴포넌트에서 다양한 기능을 사용할 수 있게 만들어준 라이브러리 (React 16.8버전 이후, 현재 20여개정도)

State란?

React 컴포넌트

State



@한 입 크기로 잘라먹는 리액트

State



State



State를 갖는
React 컴포넌트

상태변화 감지!

State

OFF → ON

렌더링 결과

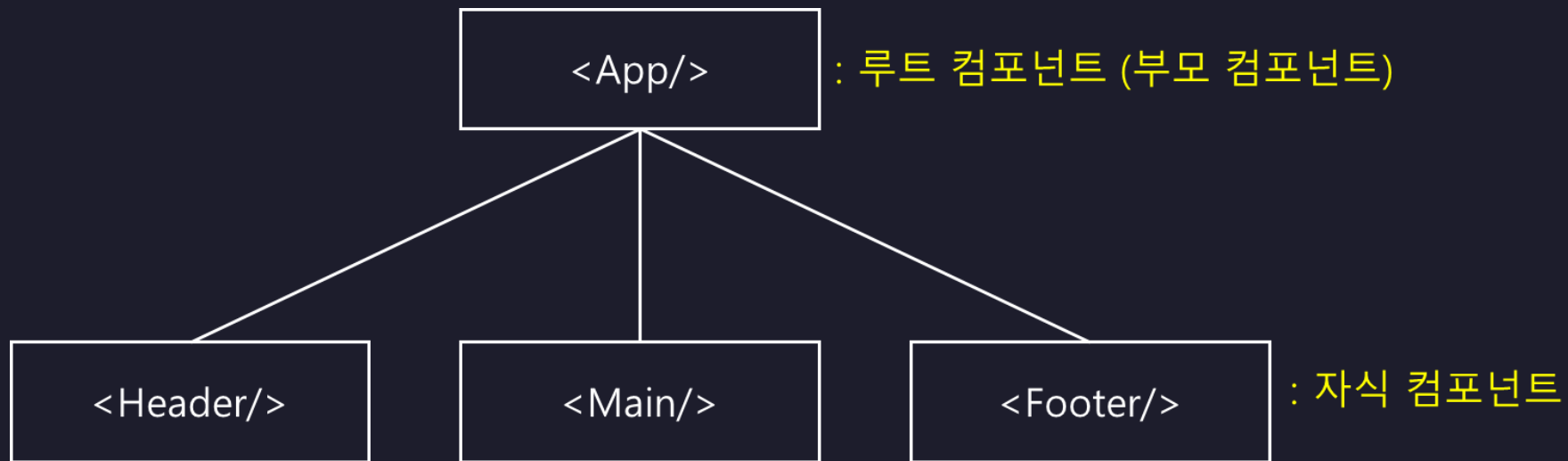
@한 입 크기로 잘라먹는 리액트

리 렌더(Re-Render)

리 렌더링 (Re-Rendering)

컴포넌트의 리렌더링이 되는 경우

1. 본인의 state값이 변경되었을 때
2. 부모로부터 전달받는 Props의 값이 변경되었을 때
3. 부모 컴포넌트가 리렌더링이 되면 자식들도 리렌더링이 됨



한입 크기로 잘라먹는

카운터 앱

Simple Counter

현재 카운트 :

0

-1

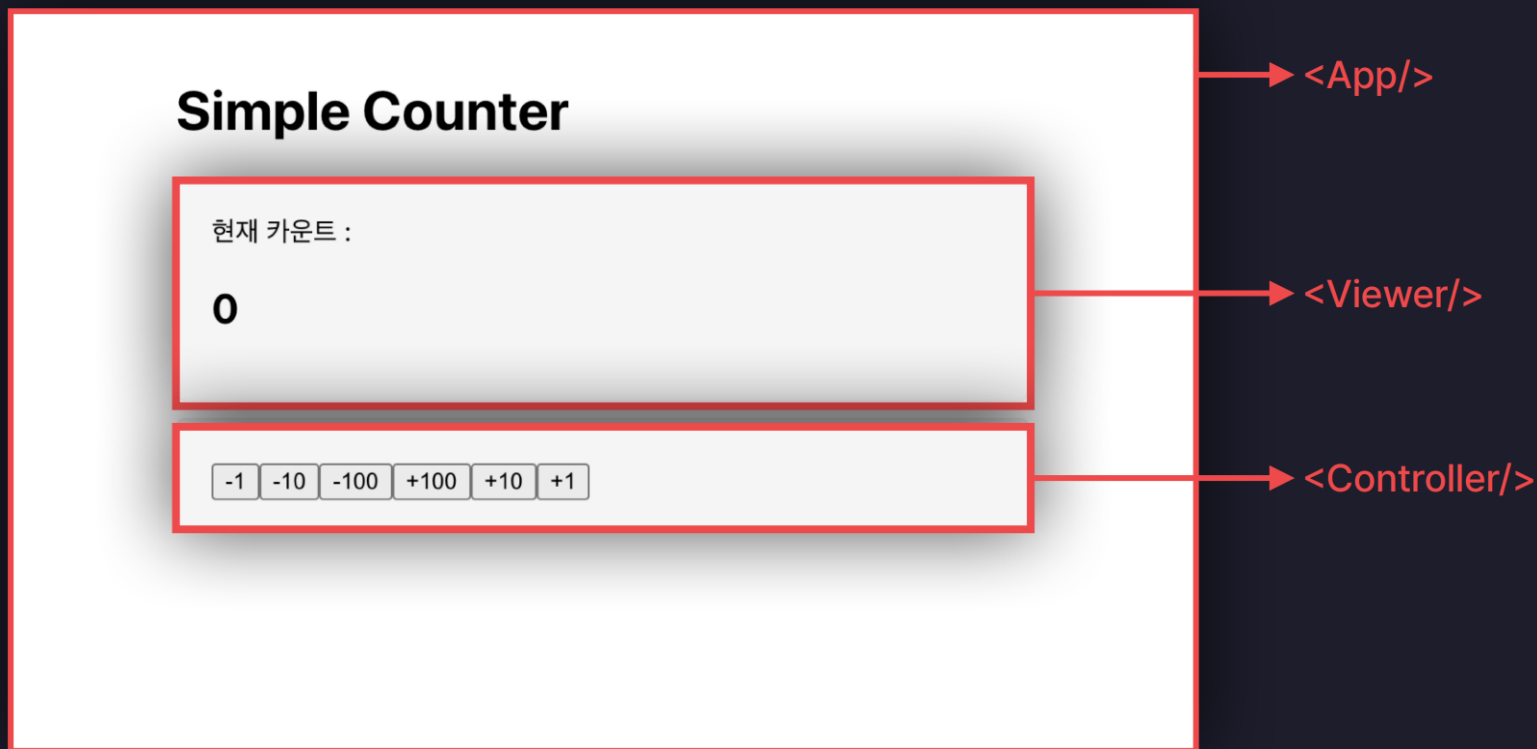
-10

-100

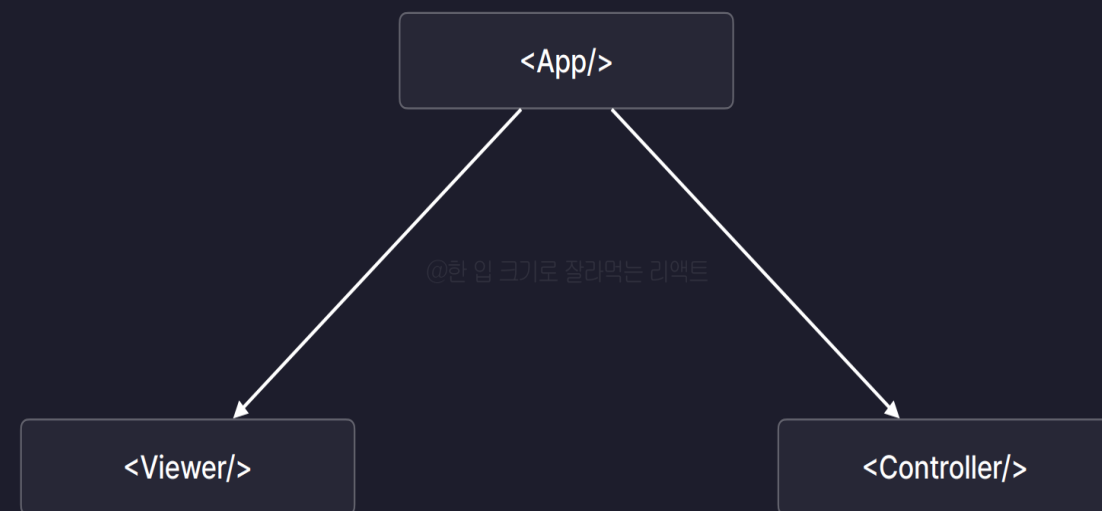
+100

+10

+1



Counter App의 컴포넌트 계층 구조



※ 상태 끌어올리기 (Lifting State Up)

두 개 이상의 자식 컴포넌트가 동일한 데이터를 공유해야 할 때 가장 가까운 공통 부모 컴포넌트로 "끌어올림".
즉, 상태를 자식 컴포넌트에서 부모 컴포넌트로 이동시키는 것

`[count, setCount]` 선언은 어디에?

useRef란?

useRef

새로운 Reference 객체를 생성하는 기능

@한 입 크기로 잘라먹는 리액트

```
const refObject = useRef()
```

컴포넌트 내부의 변수 { current : undefined }

컴포넌트 내부에서 사용할 변수를 생성

컴포넌트가 렌더링하는 특정 DOM 요소에 접근

useRef란?

```
const ref0bject = useRef()
```

```
<div>  
  <textarea  
    ref={ref0bject}  
    name="bio"  
    value={input.bio}  
    onChange={onChange}  
  />  
</div>
```

특정 DOM 요소에 접근

@한 입 크기로 잘라먹는 리액트

[요소 조작]
Focus!

useRef란?

useRef

Reference 객체를 생성

@한 입 크기로 잘라먹는 리액트

컴포넌트 내부의 변수로 활용 가능

어떤 경우에도 리렌더링을 유발하지 않음

useState

State를 생성

컴포넌트 내부의 변수로 활용 가능

값이 변경되면 컴포넌트 리렌더링

컴포넌트 내부에서 사용할 변수를 생성한다는 점에서는 동일

그러나 useRef로 생성한 변수는 값이 바뀌더라도 리렌더링을 유발하지 않음

라이프사이클(LifeCycle) = 생애 주기



*출처 : 국민재난안전포털

리액트 컴포넌트의 라이프사이클



Mount

Like. 탄생

- 컴포넌트가 탄생하는 순간
- 화면에 처음 렌더링 되는 순간

"A 컴포넌트가 Mount 되었다!"

⇒ A 컴포넌트가 화면에 처음으로 렌더링 되었다.

서버에서 데이터를 불러오는 작업



Update

Like. 변화

- 컴포넌트가 다시 렌더링 되는 순간
- 리렌더링 될 때를 의미

"A 컴포넌트가 업데이트 되었다!"

⇒ A 컴포넌트가 리렌더링 되었다

어떤 값이 변경되었는지 콘솔에 출력



UnMount

Like. 죽음

- 컴포넌트가 화면에서 사라지는 순간
- 렌더링에서 제외 되는 순간을 의미

"A 컴포넌트가 언마운트 되었다!"

⇒ A 컴포넌트가 화면에서 사라졌다

컴포넌트가 사용하던 메모리 정리

리액트 컴포넌트의 라이프사이클

라이프사이클 제어



useEffect

Side Effect!



Mount

서버에서 데이터를 불러오는 작업

@한 입 크기로 잘라먹는 리액트



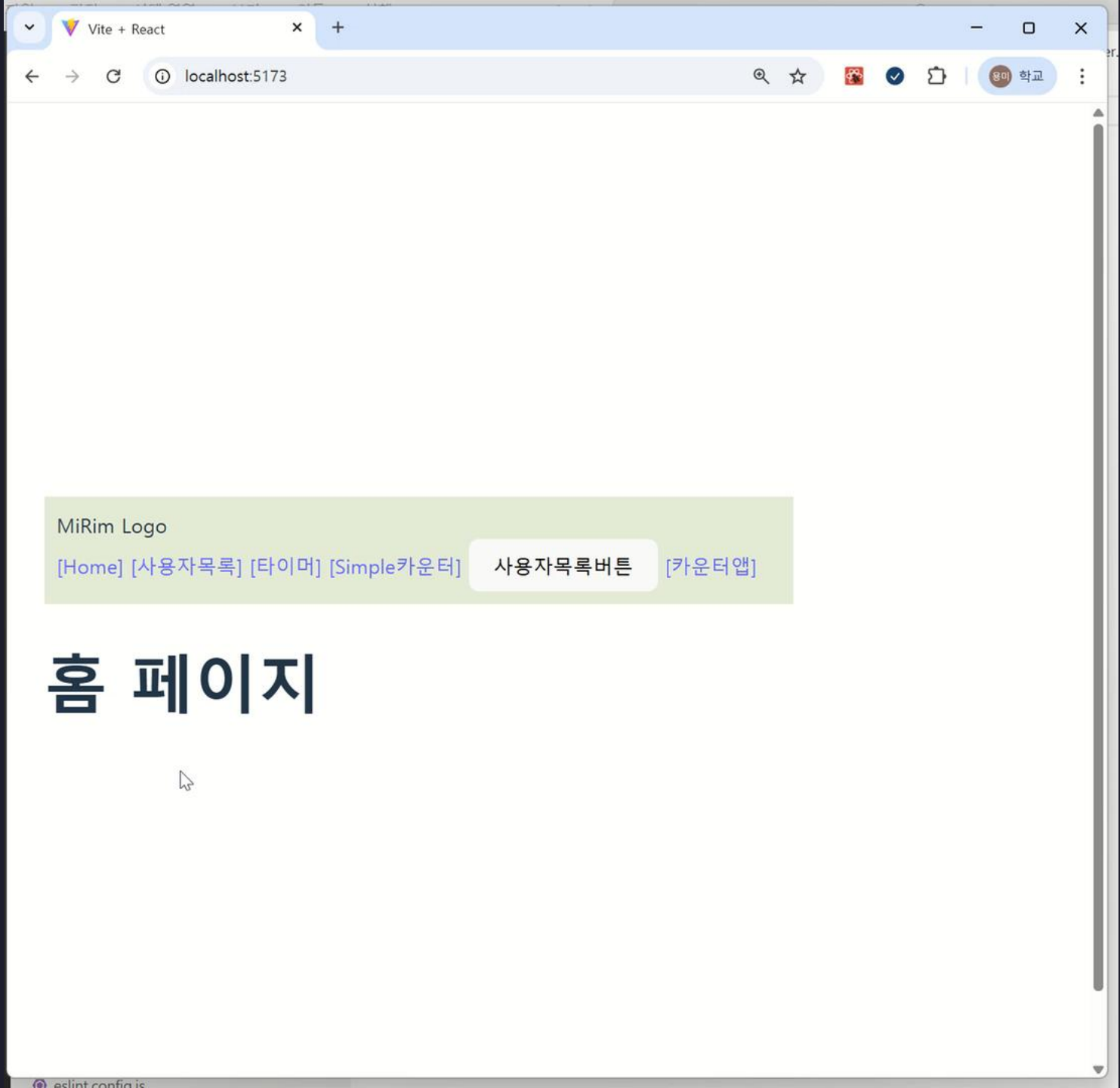
Update

어떤 값이 변경되었는지 콘솔에 출력



UnMount

컴포넌트가 사용하던 메모리 정리



한입 크기로 잘라먹는

TodoList - 프로젝트

오늘은 📅

Mon May 06 2024

새로운 Todo 작성하기 ➡

새로운 Todo...

추가

Todo List 🌱

검색어를 입력하세요

☐ React 공부하기

2024. 5. 6.

삭제

☐ 빨래 널기

2024. 5. 6.

삭제

☐ 노래 연습하기

2024. 5. 6.

삭제

Header 컴포넌트

Editor 컴포넌트

List 컴포넌트

TodoItem 컴포넌트

< 데이터 모델링 >

id	isDone	content	date
1	false	React공부하기	2024.5.6
2	false	빨래 널기	2024.5.6

⋮

⋮

⋮

⋮

오늘은 📅

Mon May 06 2024

새로운 Todo 작성하기 ➡

새로운 Todo...

추가

Todo List 🌱

검색어를 입력하세요

☐ React 공부하기

2024. 5. 6.

삭제

☐ 빨래 널기

2024. 5. 6.

삭제

☐ 노래 연습하기

2024. 5. 6.

삭제

useReducer란?

useReducer

컴포넌트 내부에 새로운 State를 생성하는 React Hook

@한 입 크기로 잘라먹는 리액트

모든 useState는 useReducer로 대체 가능

상태 관리 코드를 컴포넌트 외부로 분리할 수 있음

useState를 사용하면?

컴포넌트 내부에 상태 관리 코드를 작성해야 함



```
function App() {  
  const [todos, setTodos] = useState(mockData);  
  const idRef = useRef(3);  
  
  const onCreate = (content) => {  
    const newTodo = {  
      id: idRef.current++,  
      isDone: false,  
      content: content,  
      date: new Date().getTime(),  
    };  
  
    setTodos([newTodo, ...todos]);  
  };  
}
```

@한 입 크기로 잘라먹는 리액트

useReducer란?

useReducer를 사용하면?

컴포넌트 외부에 상태 관리 코드를 분리할 수 있음

@한입 크기로 잘라먹는 리액트

```
function reducer(){  
  // ...  
}
```

```
function App() {  
  const [todos ,dispatch] = useReducer(reducer);  
  
  // ...  
}
```

useState를 사용하면?

컴포넌트 내부에 상태 관리 코드를 작성해야 함

@한입 크기로 잘라먹는 리액트

```
function App() {  
  const [todos, setTodos] = useState(mockData);  
  const idRef = useRef(3);  
  
  const onCreate = (content) => {  
    const newTodo = {  
      id: idRef.current++,  
      isDone: false,  
      content: content,  
      date: new Date().getTime(),  
    };  
  
    setTodos([newTodo, ...todos]);  
  };  
}
```

Context란?

Props의 단점: Props Drilling

React Context

컴포넌트간의 데이터를 전달하는 또 다른 방법

@한 입 크기로 잘라먹는 리액트

기존의 Props가 가지고 있던 단점을 해결할 수 있음



Props Drilling
(프롭스 드릴링)

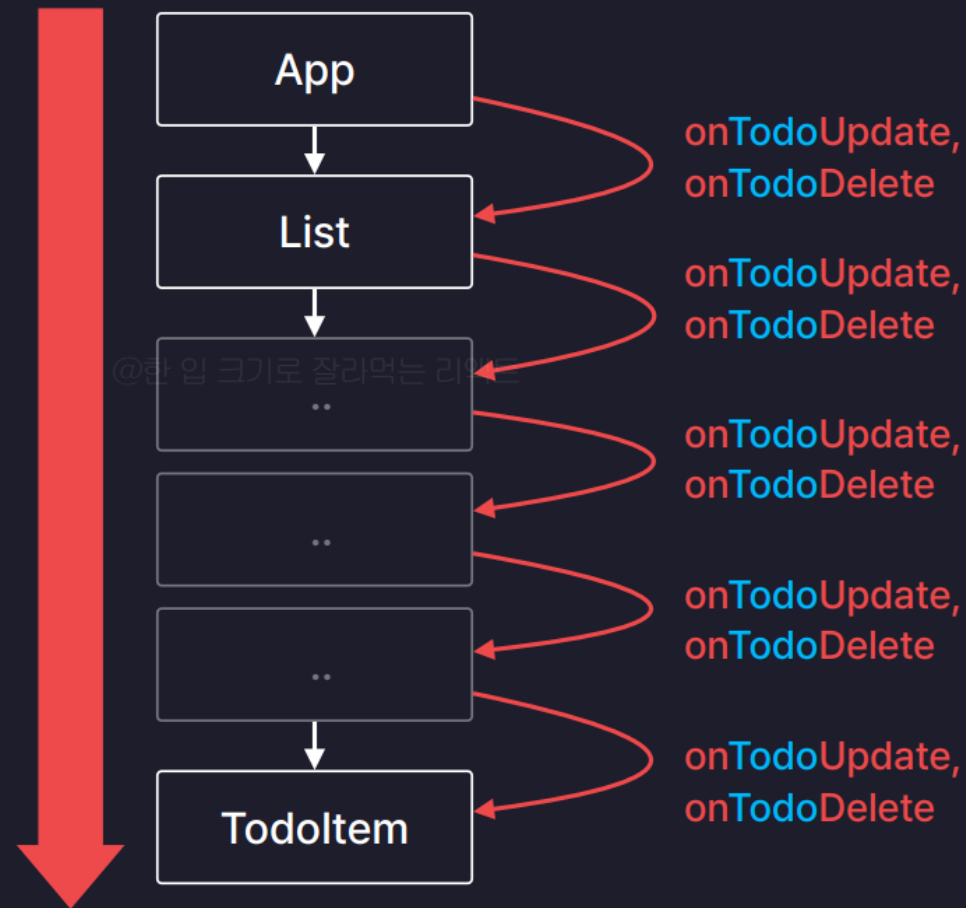
테마, 사용자 정보, 언어 설정처럼
여러 사용되는 데이터를 단계별로 props중첩된 컴포넌트에서 공통으로 전달하지 않고,
컴포넌트 트리 어디서든 접근할 수 있게 전역 변수처럼 제공하는 데 유용하며,
이는 코드 가독성과 유지보수성을 높여 줌.

✓ 사용 예

- ❖ **테마 관리:** 다크 모드/라이트 모드 설정.
- ❖ **사용자 인증:** 현재 로그인된 사용자 정보.
- ❖ **언어 설정:** 앱의 표시 언어

✓ **성능:** Context의 value가 변경되면 Provider 하위의 모든 컴포넌트가 리렌더링 될 수 있어 성능 저하의 원인

✓ **최적화:** React.memo나 useMemo 등을 활용하여 불필요한 리렌더링 최소화



```
function App() {
  return (
    <AwesomeComponent value="Hello World" />
  )
}
```

```
function AwesomeComponent({ value }) {
  return (
    <div>
      <FirstComponent value={value} />
      <SecondComponent value={value} />
      <ThirdComponent value={value} />
    </div>
  )
}
```

```
function FirstComponent({ value }) {
  return (
    <div>First Component says: "{value}"</div>
  )
}
```

```
function SecondComponent({ value }) {
  return (
    <div>Second Component says: "{value}"</div>
  )
}
```

```
function ThirdComponent({ value }) {
  return (
    <div>Third Component says: "{value}"</div>
  )
}
```

```
import { createContext, useContext } from 'react';
const MyContext = createContext();
```

```
function App() {
  return (
    <MyContext.Provider value="Hello World">
      <AwesomeComponent />
    </MyContext.Provider>
  );
}
```

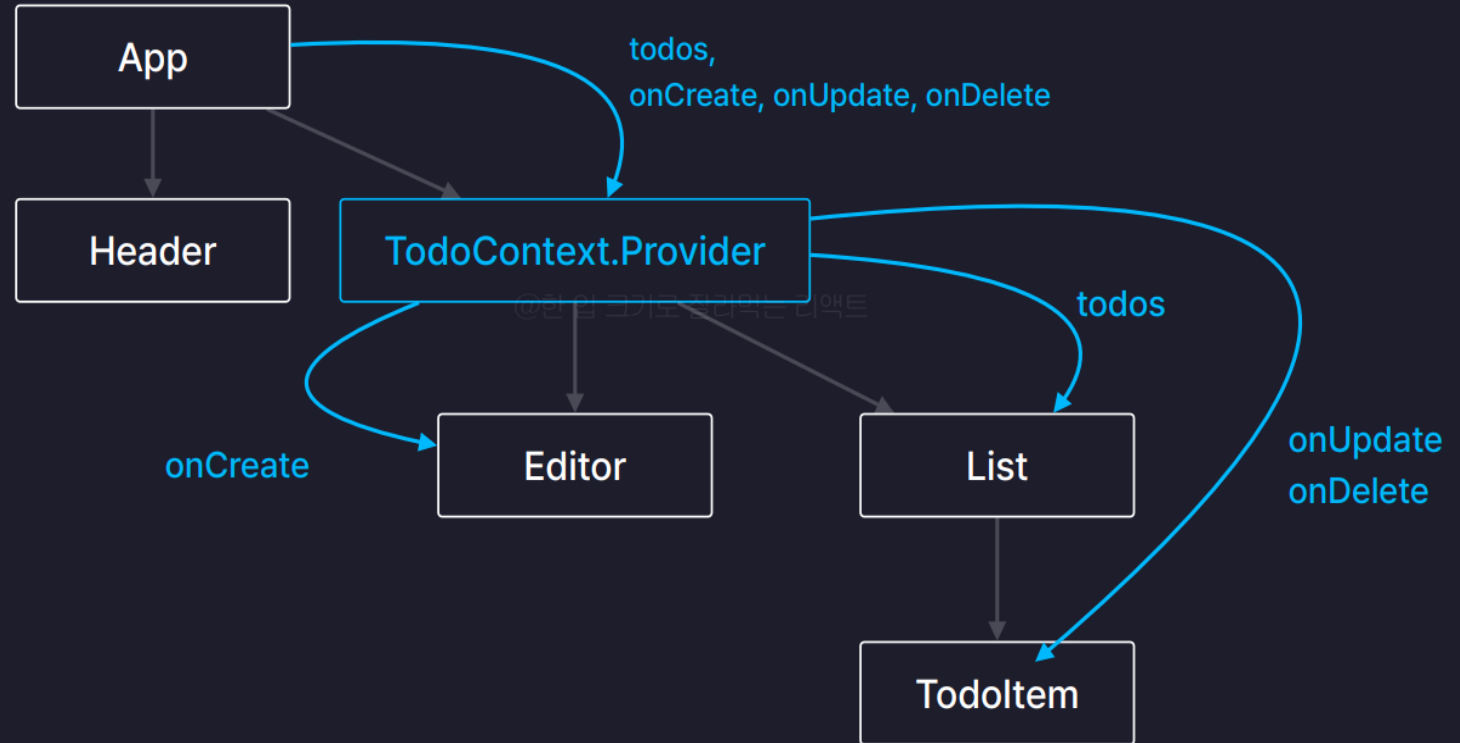
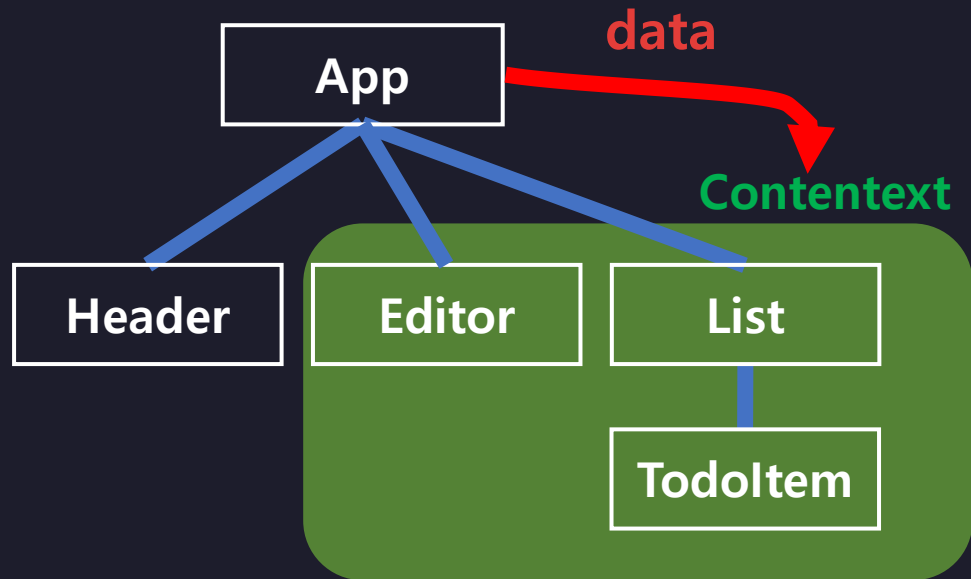
```
function AwesomeComponent() {
  return (
    <div>
      <FirstComponent />
      <SecondComponent />
      <ThirdComponent />
    </div>
  );
}
```

```
function FirstComponent() {
  const value = useContext(MyContext);
  return <div>First Component says: "{value}"</div>;
}
```

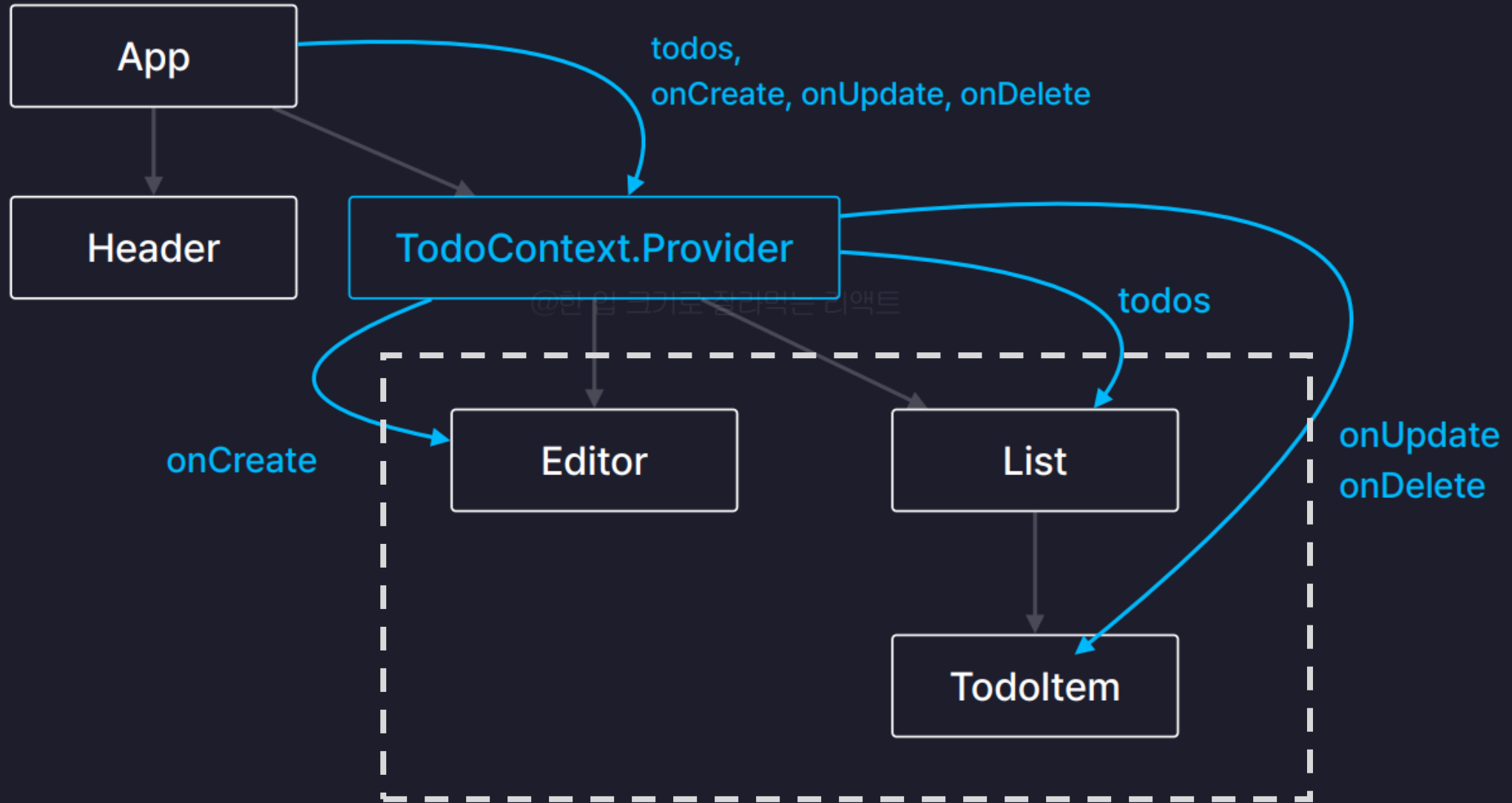
```
function SecondComponent() {
  const value = useContext(MyContext);
  return <div>Second Component says: "{value}"</div>;
}
```

```
function ThirdComponent() {
  const value = useContext(MyContext);
  return <div>Third Component says: "{value}"</div>;
}
```

Context란?

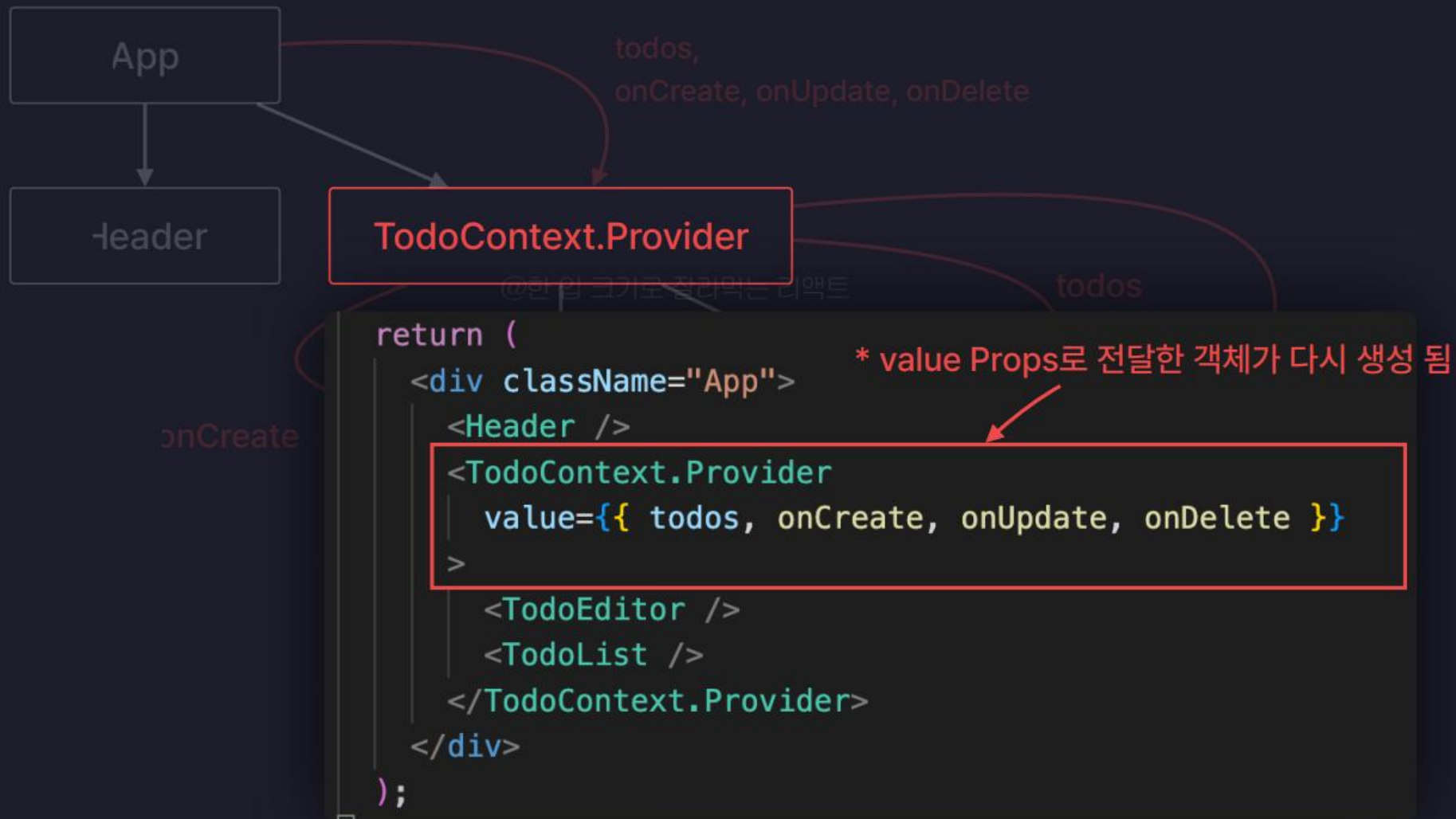


Context란?



Context란?

왜 문제가 발생했을까?



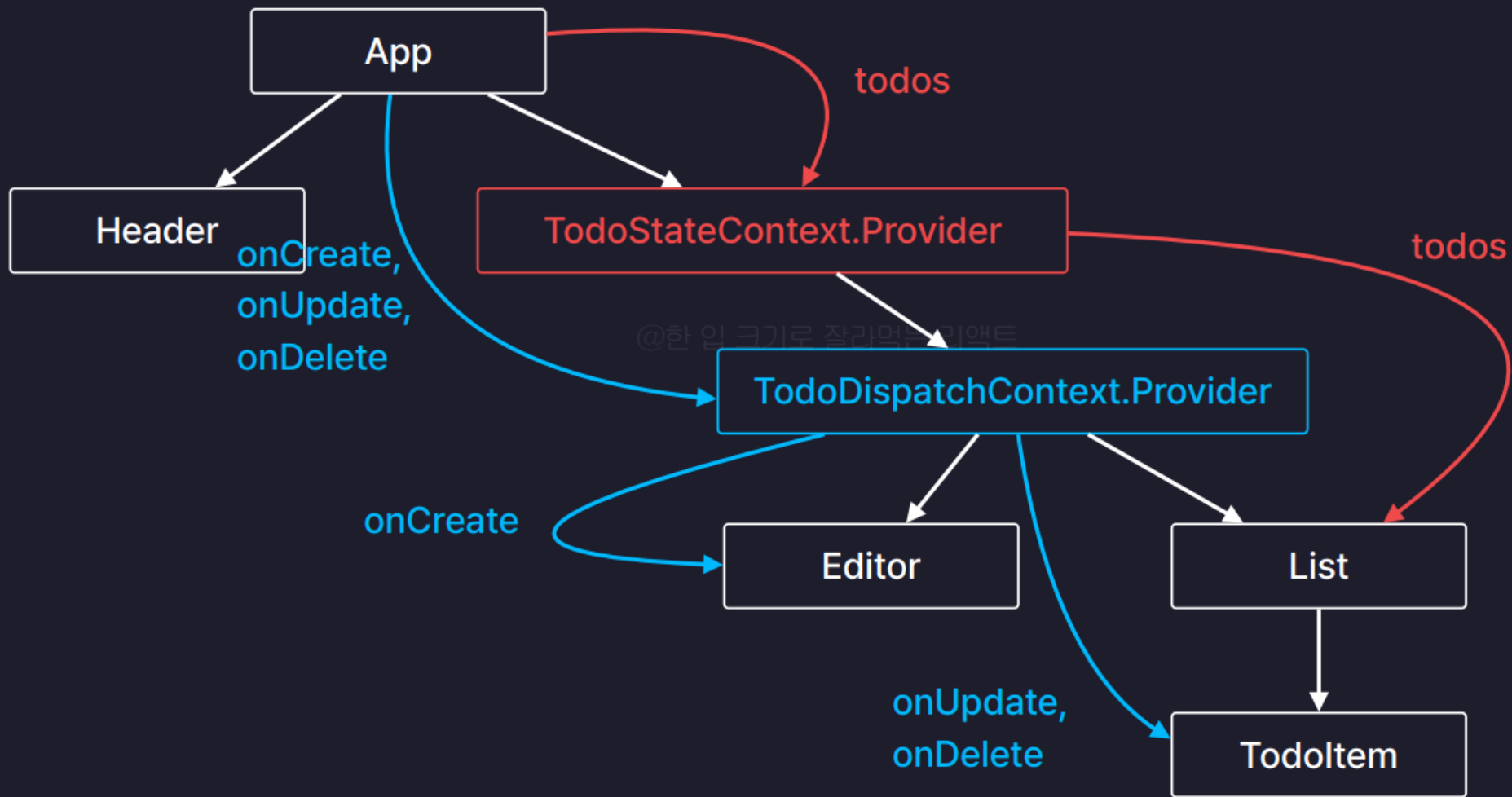
Context란?

어떻게 해결할 수 있을까?



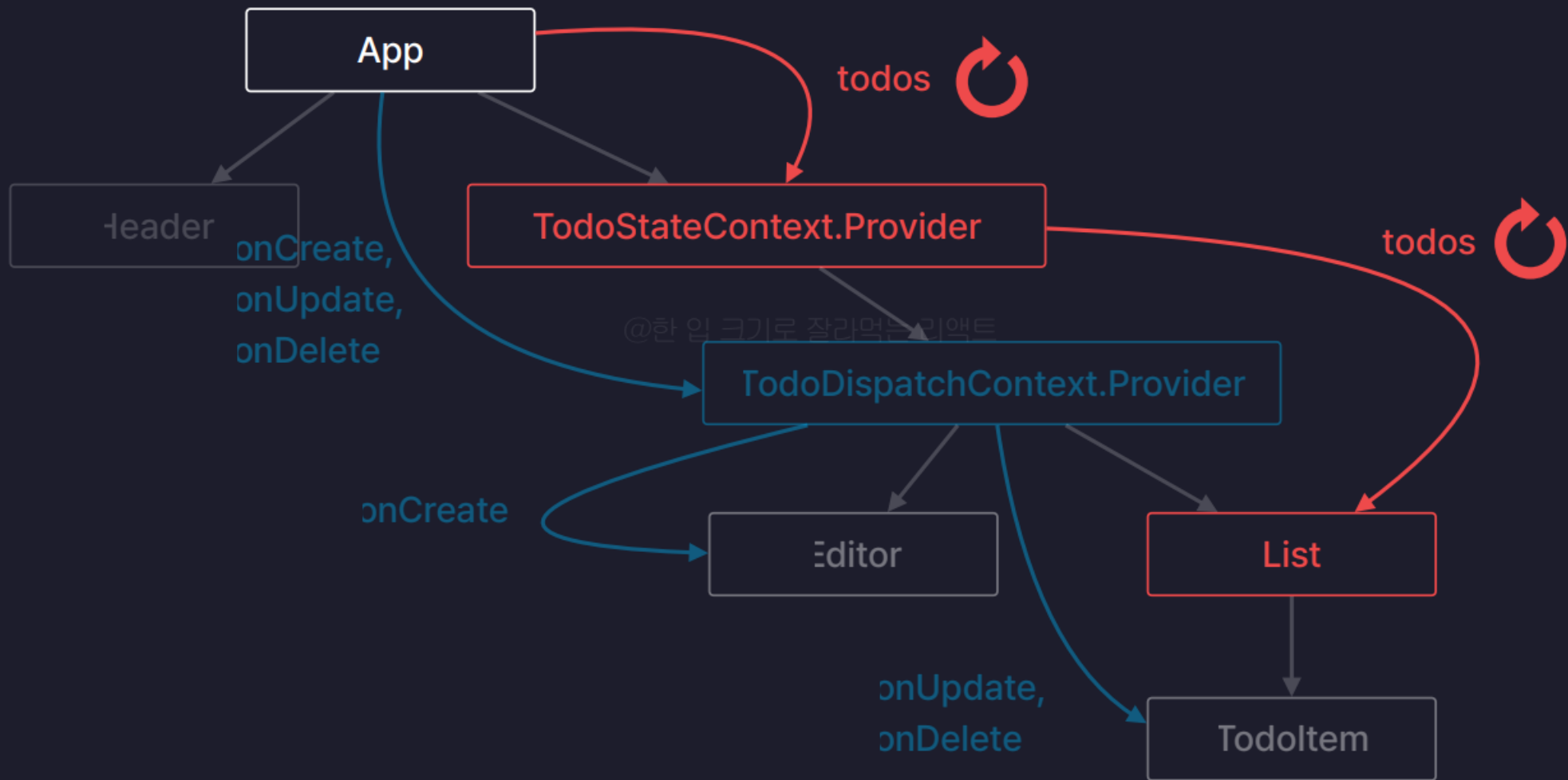
Context란?

어떻게 해결할 수 있을까?



Context란?

어떻게 해결할 수 있을까?



한입 크기로 잘라먹는

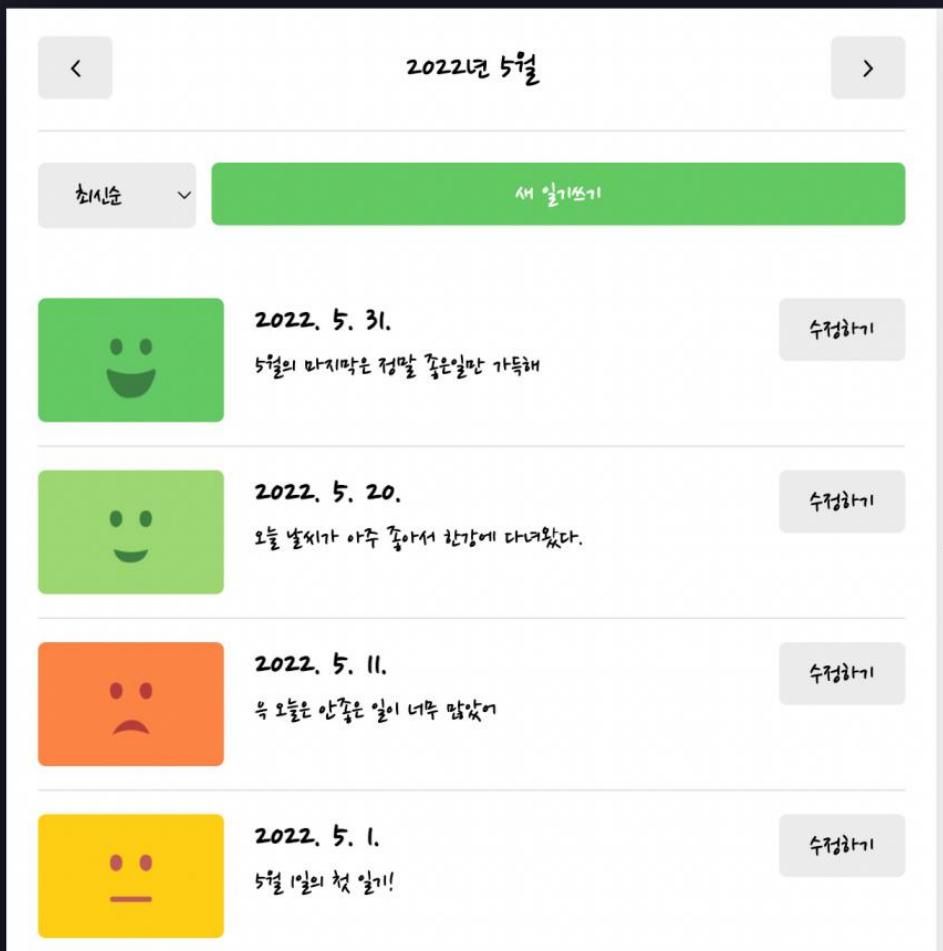
프로젝트



Final Project. 감정 일기장

감정 일기장 소개

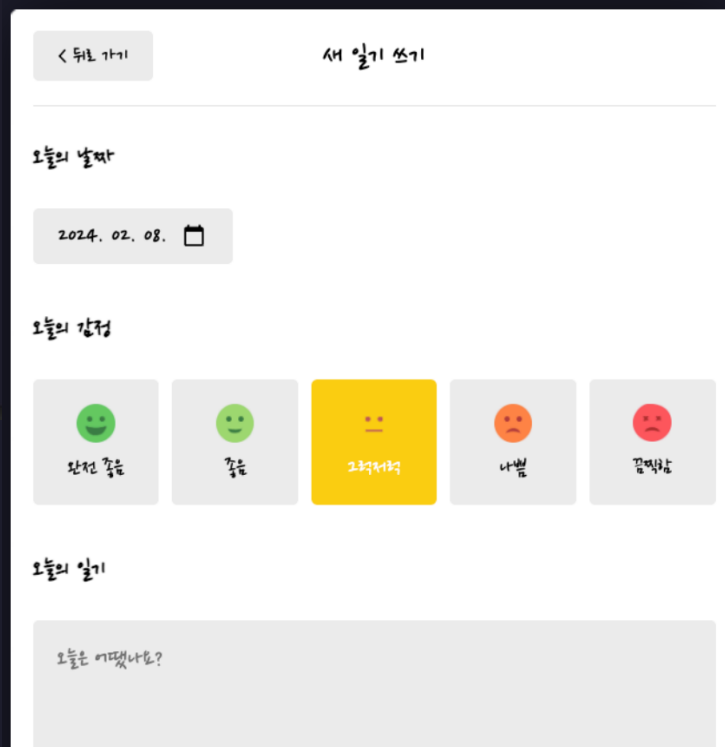
<https://emotiondiary-winterlood.web.app/>



감정 일기장

- 일기를 작성하는 서비스
- 감정까지 함께 기록 할 수 있음

리먹는 리액트



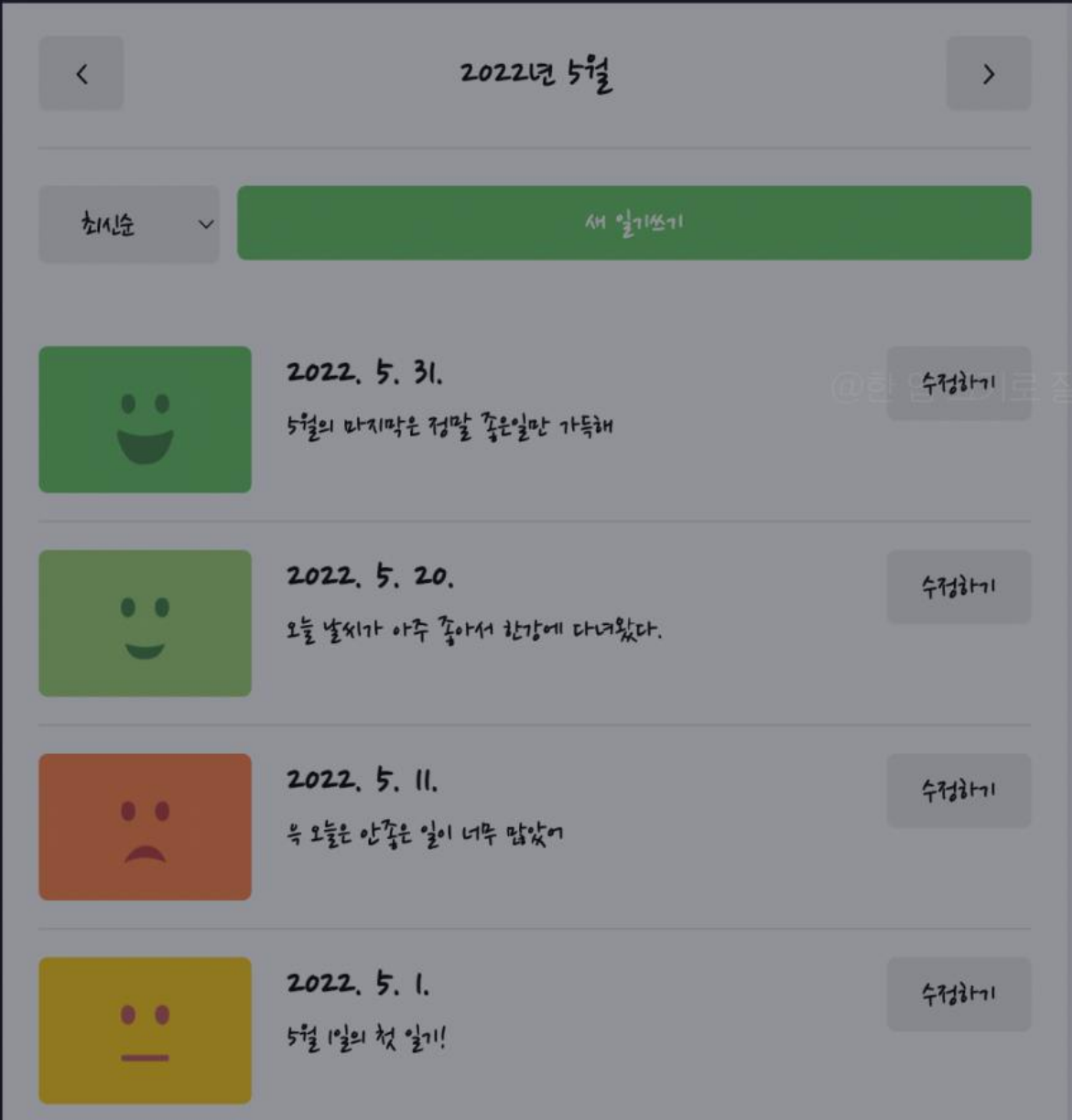
새로운 일기를 작성하는 페이지

/new 라는 경로를 가짐

- 이렇듯 감정 일기장은 여러개의 페이지로 이루어짐

리먹는 리액트

감정 일기장 프로젝트의 목표



- 외부 폰트 사용법
이미지 사용법 (+최적화)
- 다양한 페이지를 제공하는 방법
- 공통 컴포넌트로 UI 요소 모듈화
- 복잡한 데이터를 다루는 방법
- 리액트 앱을 실제로 배포하는 방법
- ... 기타 등등

페이지 라우팅(Page Routing)이란?

페이지 라우팅(Page Routing)

index

<

2022년 5월

>

최세윤

▼

새 일기쓰기



2022. 5. 31.
5월의 마지막은 정말 좋은날만 가득해

수정하기



2022. 5. 20.
오늘 날씨가 아주 좋아서 한강에 다녀왔다.

수정하기



2022. 5. 11.
음 오늘은 안좋은 일이 너무 많았어

수정하기



2022. 5. 1.
5월 1일의 첫 일기!

수정하기

new

< 뒤로 가기

새 일기 쓰기

오늘의 날짜

2024. 02. 08. 

오늘의 감정

 완전 좋음

 좋음

 그저저지

 나쁨

 끔찍함

오늘의 일기

오늘은 어땠나요?

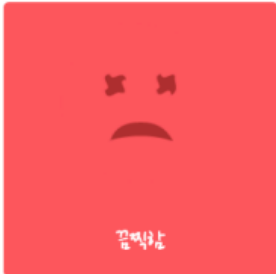
diary

< 뒤로 가기

2024-02-08 기록

수정하기

오늘의 감정



오늘의 일기

세상에서 가장 인기 있는 벌레는? 스파벅스! 총을 대충 쏘면? 설렁탕!

페이지 라우팅의 원리



Multi Page Application (MPA)

애플리케이션이 여러개의 페이지를 가지고 있음

많은 서비스가 사용하는 전통적인 방식

서버 사이드 렌더링
Server Side Rendering

페이지 라우팅의 원리

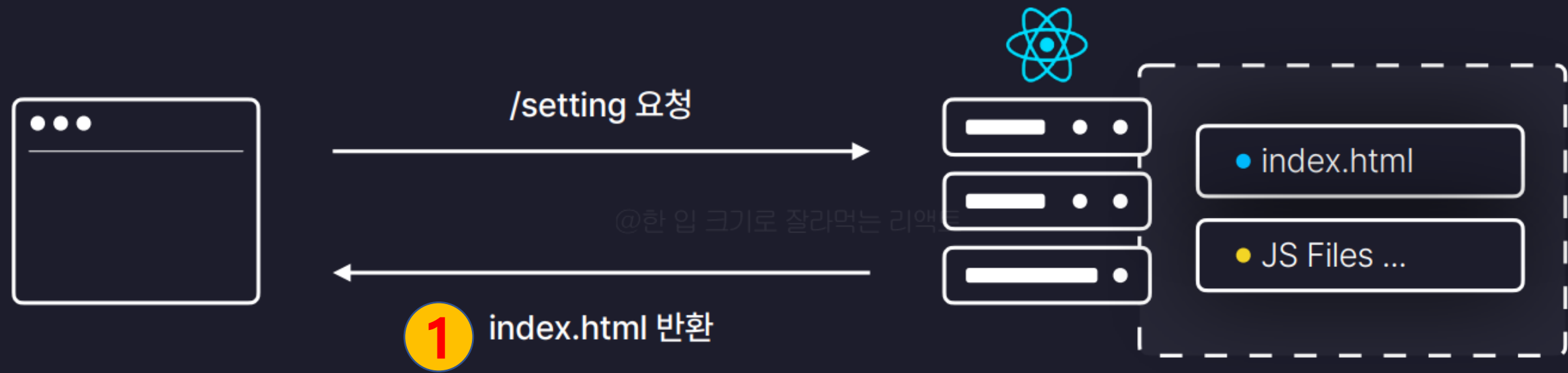
SSR의 장점

- **빠른 초기 로딩 속도:** SSR은 서버 측에서 데이터를 받아와 렌더링하기 때문에 초기 로딩 속도가 빠르다.
- **SEO에 유리함:** SSR에서는 초기 HTML 파일에 콘텐츠가 포함되어 있기 때문에 검색 엔진이 페이지를 크롤링할 때 콘텐츠를 인식할 수 있다.
- **높은 보안성:** SSR에서는 서버 측에서 데이터를 처리하기 때문에 보안성이 높다. 클라이언트 측에서 데이터를 조작하거나 해킹 등의 공격이 어려워진다.
- **서버 부하를 분산 가능:** SSR에서는 서버 측에서 데이터를 처리하기 때문에 서버 부하가 분산될 수 있다.

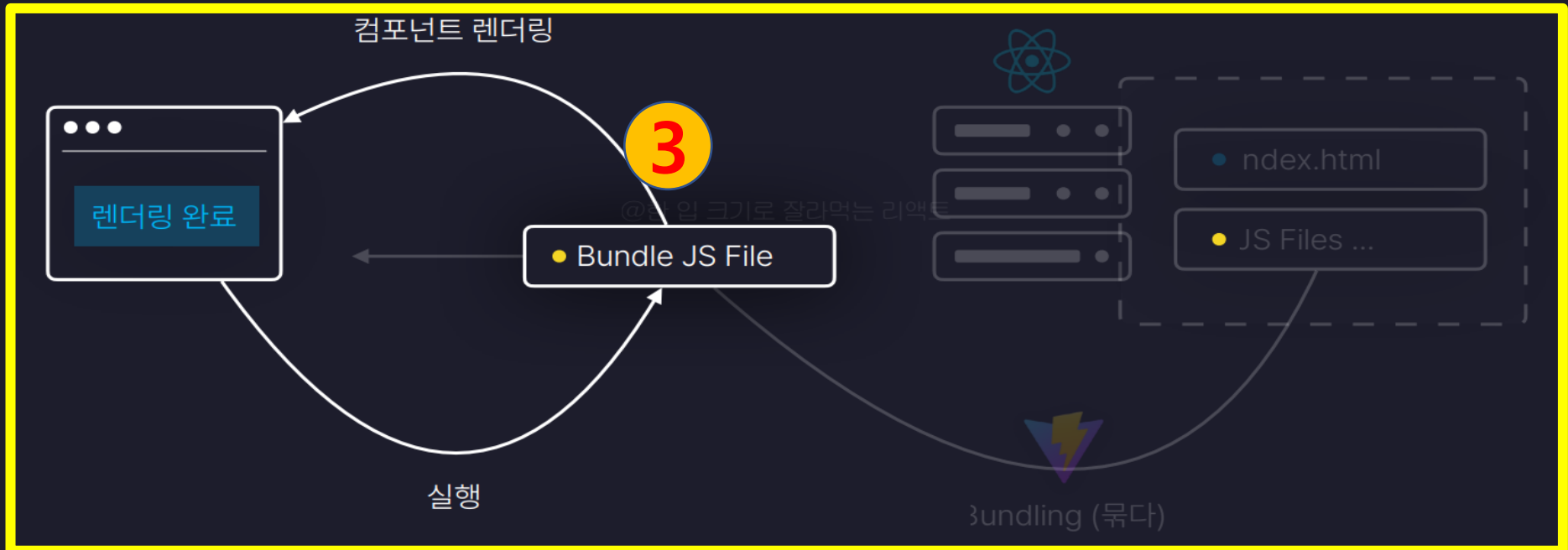
SSR의 단점

- **느린 페이지 이동 속도:** SSR은 페이지 이동할 때마다 서버에서 데이터를 받아와 렌더링해야 하기 때문에 페이지 이동 속도가 느리다.
- **낮은 사용자 경험:** SSR에서는 페이지 이동 속도가 느리기 때문에 사용자 경험이 낮을 수 있다.
- **어려운 캐싱:** SSR에서는 캐시를 사용하는 것이 어려울 수 있다. 매번 새로운 데이터를 불러와야 하기 때문에 서버 부하가 증가할 수 있다.
- **낮은 개발 난이도:** SSR은 일반적인 웹 개발 방식과 유사하기 때문에 개발 난이도가 낮다. 하지만, 대규모의 복잡한 웹 애플리케이션에서는 추가적인 복잡성이 요구될 수 있다.

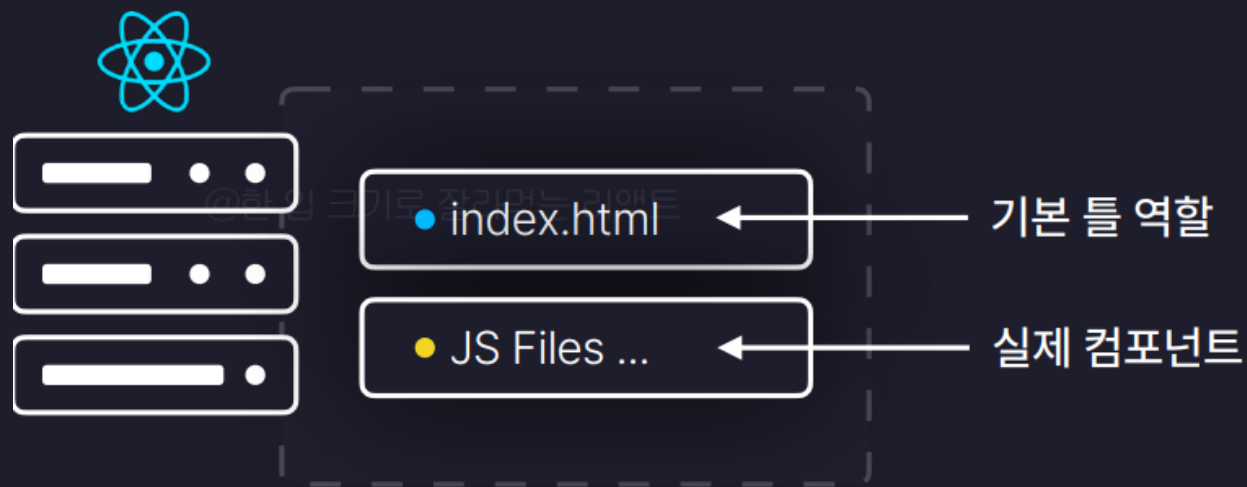
페이지 라우팅의 원리



페이지 라우팅의 원리



페이지 라우팅의 원리



SPA(Single Page Application)

React App이 채택한 방식

클라이언트 사이드 렌더링
Client Side Rendering

페이지 라우팅의 원리

CSR의 장점

- **높은 사용자 경험:** CSR은 클라이언트 측에서 데이터를 받아와 렌더링하기 때문에 페이지 이동 속도가 빠르고, 사용자 경험이 높다.
- **캐싱 가능:** 클라이언트에서 캐시를 사용하여 적은 양의 데이터만 다시 요청하여 처리하는 것이 가능하다.
- **적은 서버 부하:** CSR에서는 클라이언트에서 데이터를 처리하기 때문에 서버 부하가 적다.

CSR의 단점

- **느린 초기 로딩 속도:** CSR은 페이지를 렌더링하기 위해 필요한 JavaScript 파일을 먼저 다운로드한 후에 데이터를 받아와야 한다. 이로 인해 초기 로딩 속도가 느리게 느껴질 수 있다.
- **SEO에 불리함:** CSR에서는 초기 HTML 파일에 콘텐츠가 포함되어 있지 않기 때문에 검색 엔진이 페이지를 크롤링할 때 콘텐츠를 인식하지 못할 수 있다.
- **낮은 보안성:** CSR에서는 클라이언트에서 데이터를 처리하기 때문에 보안성이 낮을 수 있다. 클라이언트 측에서 데이터를 조작하거나 해킹 등의 공격이 가능하다.
- **높은 개발 난이도:** CSR은 대규모의 복잡한 웹 애플리케이션에서 많이 사용되며, 이를 구현하기 위해서는 높은 개발 난이도와 복잡성이 요구된다. 특히, 상태 관리와 라우팅 등을 구현하기 위해서는 추가적인 라이브러리나 프레임워크를 사용해야 한다.

페이지 라우팅의 원리

사용 권장 예시

CSR을 사용 권장

- 네트워크가 빠를 때
- 서버의 성능이 좋지 않을 때
- 사용자에게 보여줘야 하는 데이터의 양이 많을 때(ex. 대규모의 복잡한 웹 애플리케이션)
- 메인 스크립트가 가벼울 때
- SEO(Search engine Optimization) 검색 엔진 최적화가 필요 없을 때
- 웹 어플리케이션에 사용자와 상호 작용 할 것들이 많을 때
(아예 렌더링이 되지 않아 사용자의 행동을 막는것이 경험에 오히려 유리하다)

SSR을 사용 권장

- 네트워크가 느릴 때 (CSR은 한번에 모든 것을 불러오지만 SSR는 각 페이지마다 나누어 불러오기 때문이다)
- SEO(Search engine Optimization) 검색 엔진 최적화가 필요할 때
- 최초 로딩이 빨라야 하는 사이트를 개발할 때
- 메인 스크립트가 크고 로딩이 엄청 느릴 때
(CSR은 메인스크립트가 로딩이 끝나면 API로 데이터 요청을 보내지만, SSR은 한번의 요청에 아예 렌더가 가능한 페이지가 돌아온다)
- 웹 사이트가 상호작용이 별로 없을 때

CSR과 SSR 비교 정리

	CSR	SSR
초기 로딩 속도	느림	빠름
페이지 이동 속도	빠름	느림
SEO	불리함	유리함
보안성	낮음	높음
사용자 경험	높음	낮음
서버 부하	낮음	높음
개발 난이도	높음(복잡한 SPA 구현)	낮음(일반적인 웹 개발)
캐싱	가능	불가능

페이지 라우팅 - 라우팅 설정하기



npmjs.com에 등록되어 있는 라이브러리

@한 입 크기로 잘라먹는 리액트

대다수의 리액트 앱이 사용하고 있는 대표격 라이브러리

제공하는 기능 목록

Client Side Routing
Nested Routing
Dynamic Segments
Ranked Route Matching
Active Links
Relative Links
Data Loading
Redirects
Pending Navigation UI
Skeleton UI With <Suspense>
Data Mutations
...
(이하 중략)

<설정>

1. [터미널] : `npm i react-router-dom`
2. [main.jsx] :
`import { BrowserRouter } from "react-router-dom";`
`<BrowserRouter>`
3. [App.jsx] :
`import { Routes, Route } from "react-router-dom";`
`<Routes>`
`<Route path="" element={ } />`
`</Routes>`

동적 경로(Dynamic Segments)란?

URL Parameter

/ 뒤에 아이템의 id를 명시

@한 입 크기로 잘라먹는 리액트

~/product/**1**

~/product/**2**

~/product/**3**

아이템의 id 등의 변경되지 않는 값을
주소로 명시하기 위해 사용 됨

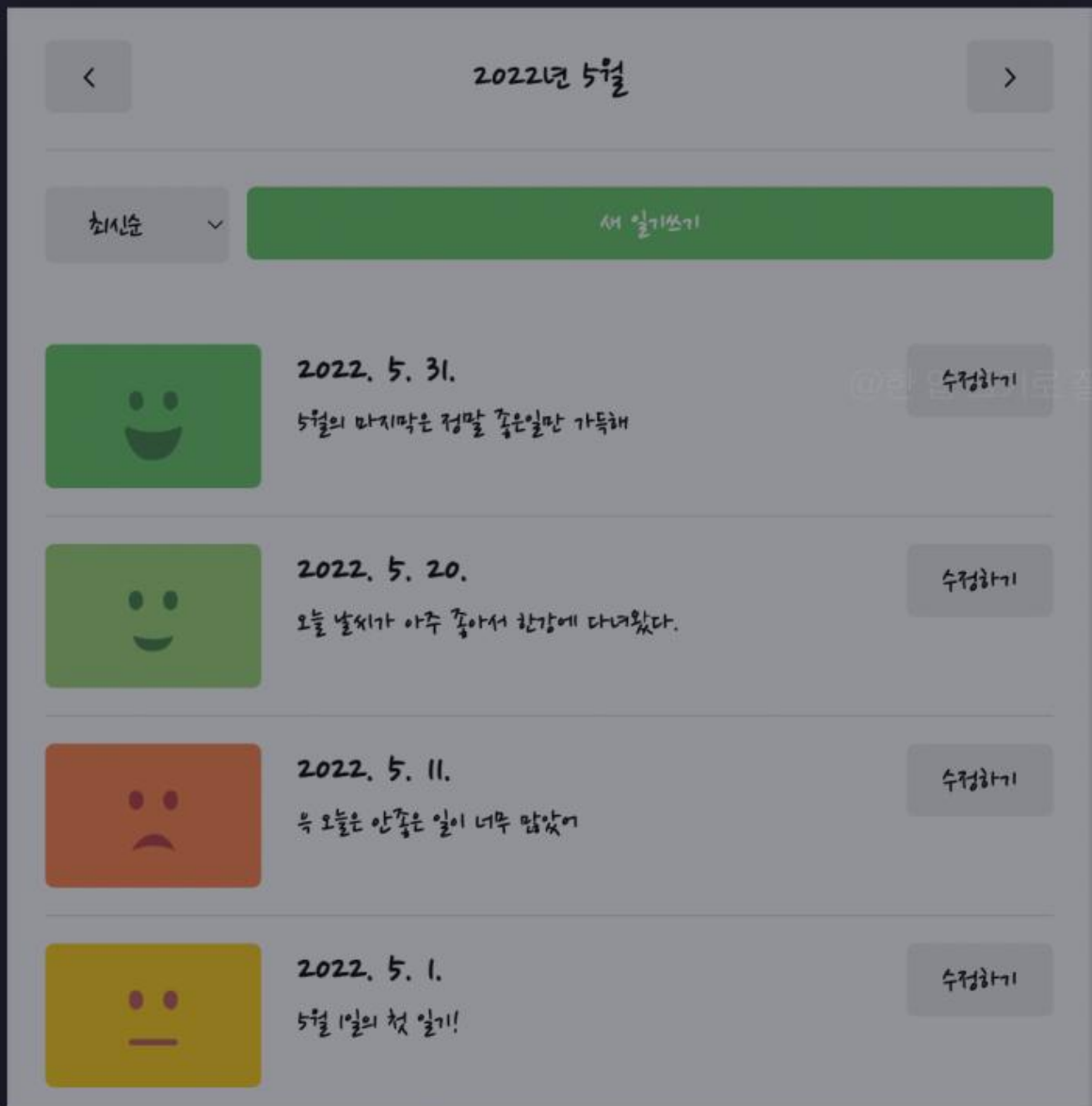
Query String

? 뒤에 변수명과 값 명시

~/search?**q=검색어**

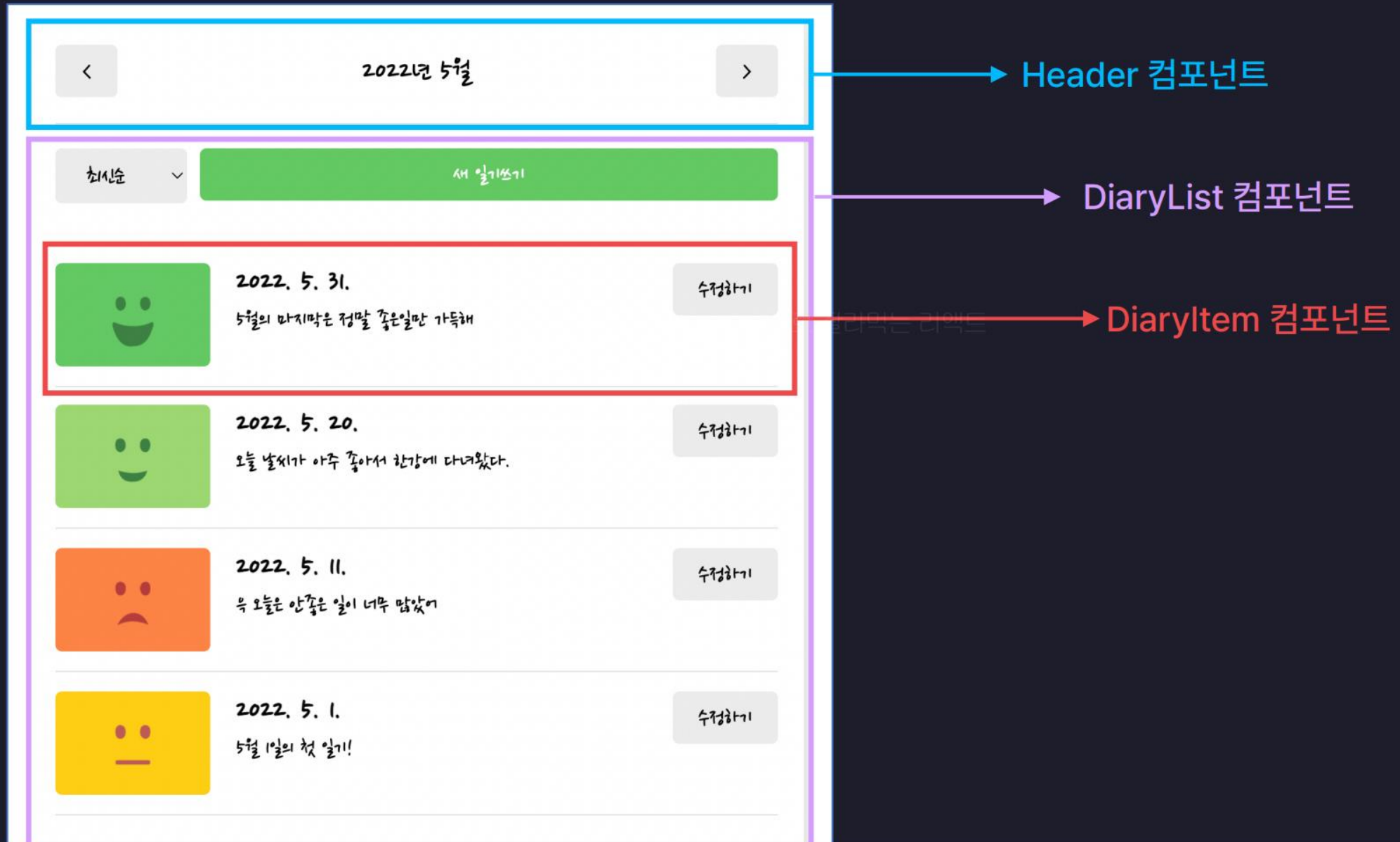
검색어 등의 자주 변경되는 값을
주소로 명시하기 위해 사용된다

감정 일기장 프로젝트의 목표

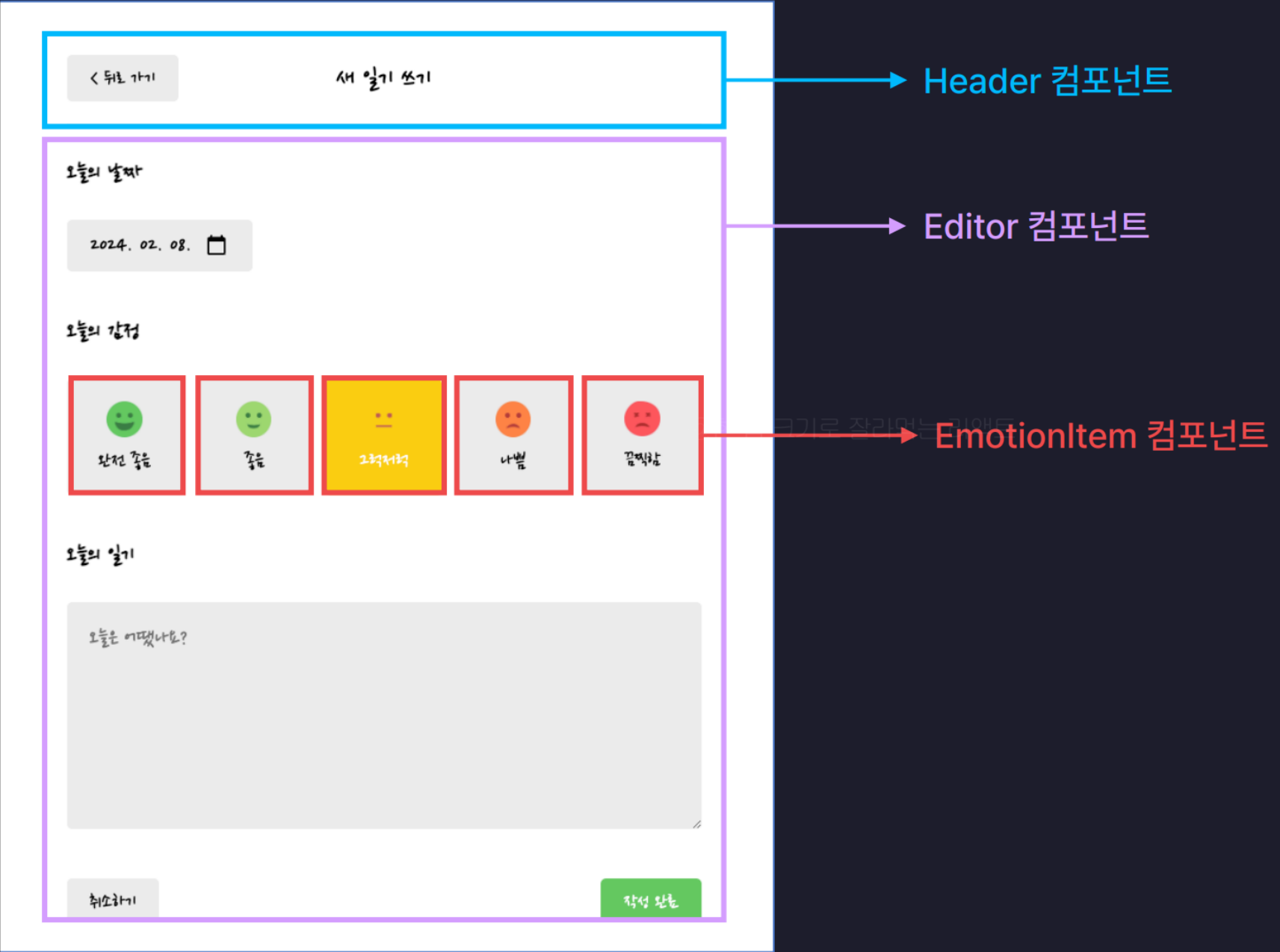


- 외부 폰트 사용법
- 이미지 사용법 (+최적화)
- 다양한 페이지를 제공하는 방법
- 공통 컴포넌트로 UI 요소 모듈화
- 복잡한 데이터를 다루는 방법
- 리액트 앱을 실제로 배포하는 방법
- ... 기타 등등

여러개의 일기를 리스트로 렌더링하는 "Home" 페이지



새로운 일기를 생성하는 "New" 페이지



기존 일기를 수정하는 "Edit" 페이지

< 뒤로 가기

일기 수정하기

삭제하기

오늘의 날짜

2024. 02. 08. 

오늘의 감정

완전 좋음

좋음

그럭저럭

나쁨

끔찍함

오늘의 일기

세상에서 가장

취소하기

작성 완료

Header 컴포넌트

Editor 컴포넌트

크기로 잘라먹는 리액트

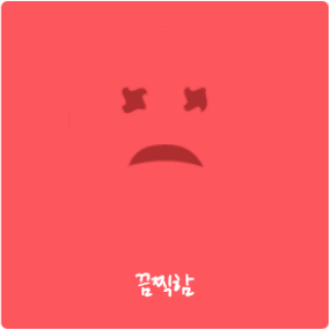
하나의 일기를 조회하는 "Diary" 페이지

< 뒤로 가기

2024-02-08 기록

수정하기

오늘의 감정



오늘의 일기

세상에서 가장 인기 있는 벌레는? 스타벅스! 총을 대충 쏘면? 설렁탕!

화장실에서 방금 나온 사람은? 일본 사람!

header 컴포넌트

Viewer 컴포넌트

한 입 크기로 잘라먹는 리액트

웹 스토리지(Web Storage)

데이터를 브라우저에 보관하는 방법, 일종의 데이터 베이스

@한 입 크기로 잘라먹는 리액트

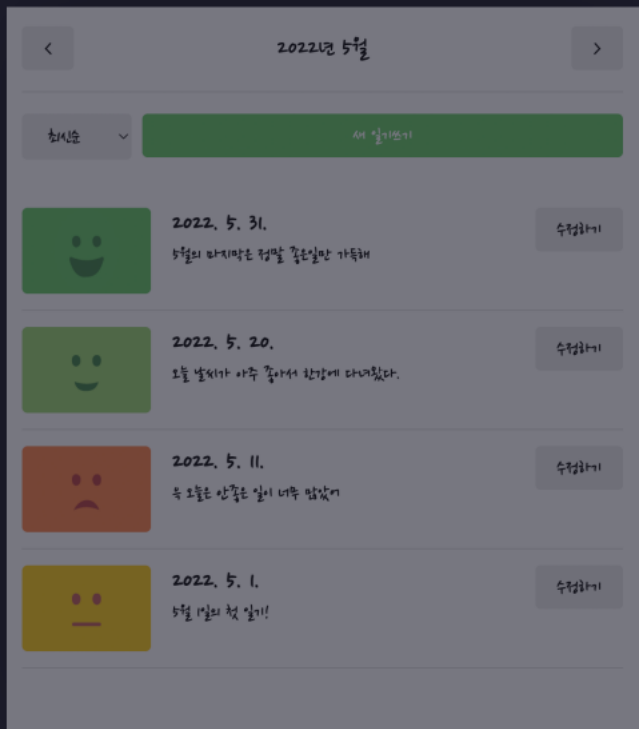
배포 준비하기

프로젝트를 배포하기 전, 타이틀, Favicon, OG 설정 진행

배포하기

리액트 앱을 배포하는 방법

웹 스토리지(Web Storage)



웹 브라우저 내장 DB

Web Storage



현재 State값 저장

@한 입 크기로 잘라먹는 리액트

저장된 State값 꺼내옴

- 웹 브라우저에 기본적으로 내장되어 있는 데이터베이스
- 별도의 프로그램 설치 필요 X, 라이브러리 설치 필요 X
- 그냥 자바스크립트 내장함수만으로 접근 가능

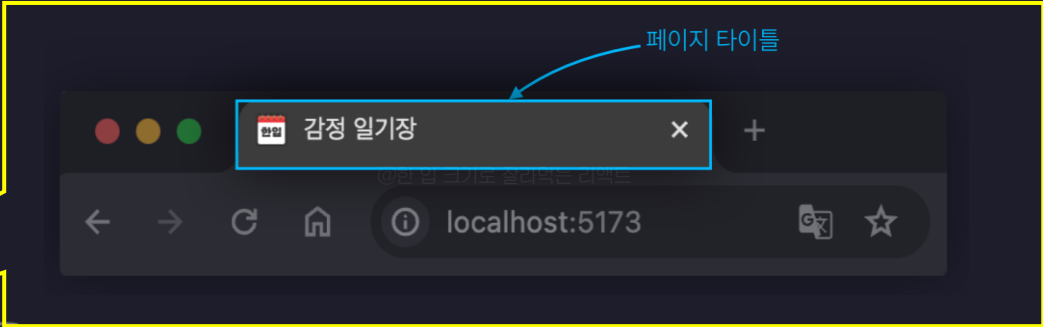
@한 입 크기로 잘라먹는 리액트

- ex)
 - 값을 저장 : `localStorage.setItem(key, value)`
 - 값을 꺼냄 : `localStorage.getItem(key)`

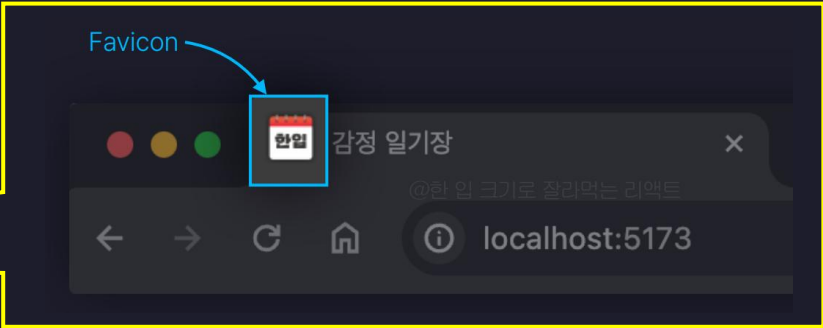
배포 준비하기

배포 준비를 위해 해야 할 작업

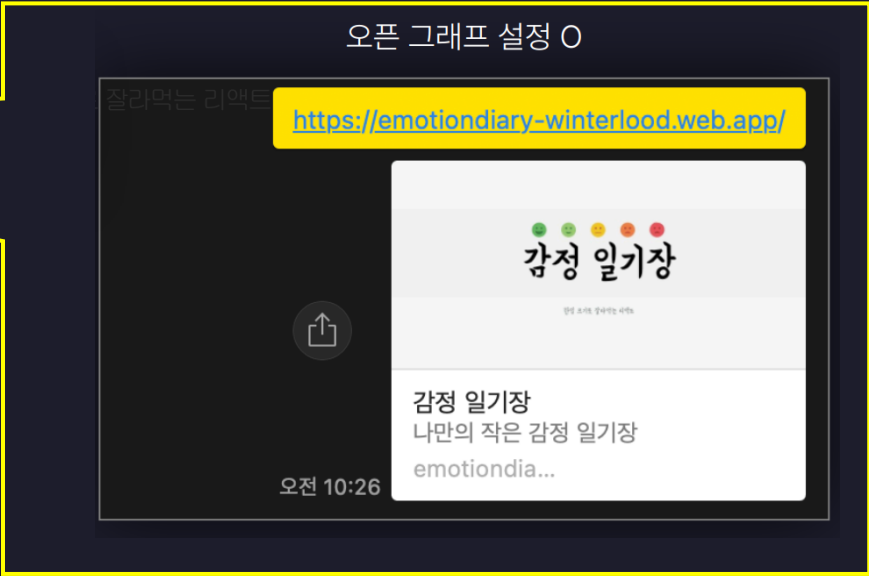
페이지 타이틀 설정하기



Favicon 설정하기



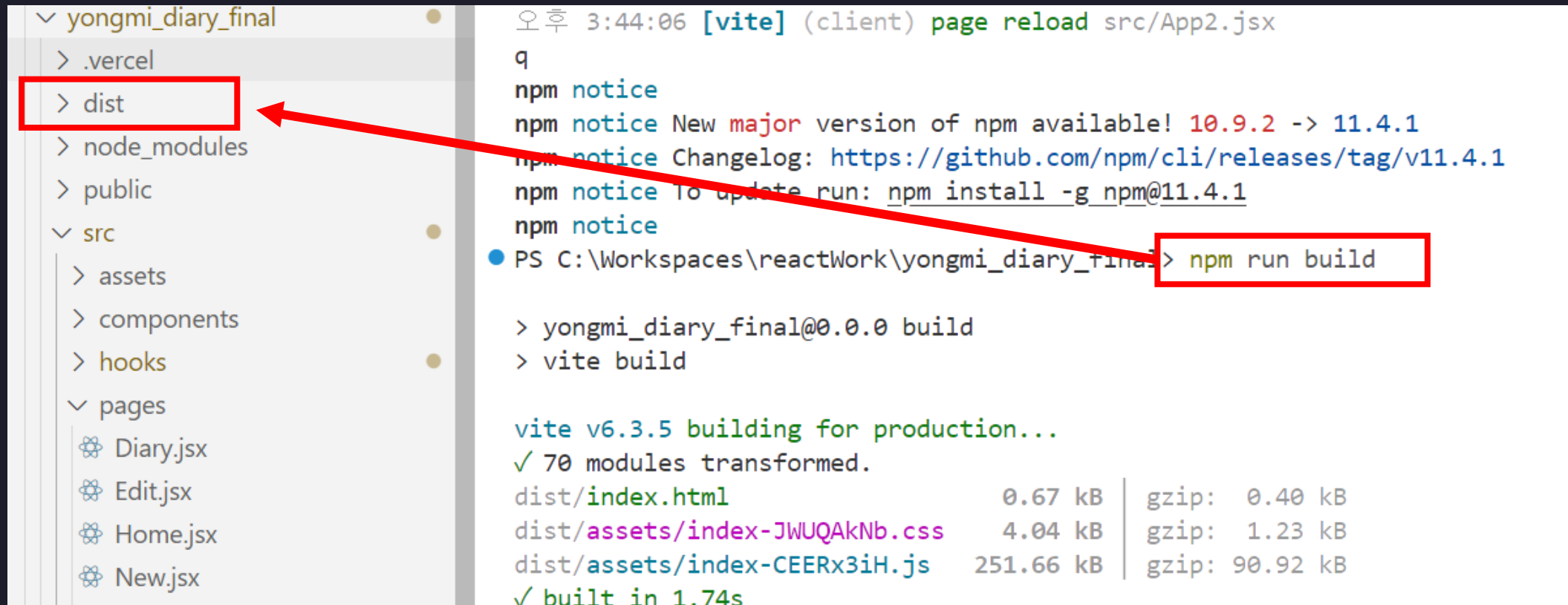
오픈 그래프 태그 설정하기



프로젝트 빌드(Build)

배포 준비하기

프로젝트 빌드



The image shows a code editor interface with a file explorer on the left and a terminal on the right. The file explorer shows a project structure for 'yongmi_diary_final'. The 'dist' folder is highlighted with a red box. The terminal shows the output of the 'npm run build' command, which includes a message about a new major version of npm available and a table of build artifacts.

File Explorer Structure:

- yongmi_diary_final
 - .vercel
 - dist
 - node_modules
 - public
 - src
 - assets
 - components
 - hooks
 - pages
 - Diary.jsx
 - Edit.jsx
 - Home.jsx
 - New.jsx

Terminal Output:

```
오후 3:44:06 [vite] (client) page reload src/App2.jsx
q
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.4.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.4.1
npm notice To update run: npm install -g npm@11.4.1
npm notice
PS C:\Workspaces\reactWork\yongmi_diary_final> npm run build

> yongmi_diary_final@0.0.0 build
> vite build

vite v6.3.5 building for production...
✓ 70 modules transformed.
dist/index.html                0.67 kB | gzip: 0.40 kB
dist/assets/index-JWUQAKNb.css 4.04 kB | gzip: 1.23 kB
dist/assets/index-CEERx3iH.js 251.66 kB | gzip: 90.92 kB
✓ built in 1.74s
```

배포하기(Deploy)

@한 일 일기부

현존 유명 클라우드 서비스



AWS



GCP



Develop. Preview. Ship.

Vercel



Netlify



Firebase

프론트엔드 개발자를 위한 클라우드 서비스

React.js의 확장판 개념인 Next.js를 개발하는 회사

https://vercel.com/

```
PS C:\Workspaces\reactWork\yongmi_diary_final> npm i -g vercel
npm warn deprecated path-match@1.2.4: This package is archived and no longer maintained. For support, visit https://github.com/expressjs/express/discussions

added 228 packages in 25s

24 packages are looking for funding
  run `npm fund` for details
PS C:\Workspaces\reactWork\yongmi_diary_final> vercel login
Vercel CLI 42.3.0
> NOTE: The Vercel CLI now collects telemetry regarding usage of the CLI.
> This information is used to shape the CLI roadmap and prioritize features.
> You can learn more, including how to opt-out if you'd not like to participate in this program, by visiting the following URL:
> https://vercel.com/docs/cli/about-telemetry
? Log in to Vercel Continue with Email
? Enter your email address: changmi29@dongyang.ac.kr
We sent an email to changmi29@dongyang.ac.kr. Please follow the steps provided inside it and make sure the security code matches Futuristic Millipede.
> Success! Email authentication complete for changmi29@dongyang.ac.kr
Congratulations! You are now logged in. In order to deploy something, run `vercel`.
💡 Connect your Git Repositories to deploy every branch push automatically (https://vercel.link/git).
PS C:\Workspaces\reactWork\yongmi_diary_final> vercel
Vercel CLI 42.3.0
? Set up and deploy "C:\Workspaces\reactWork\yongmi_diary_final"? yes
? Which scope should contain your project? Yongmi Chang's projects
? Link to existing project? no
? What's your project's name? yongmi-diary-final
? In which directory is your code located? ./
Local settings detected in vercel.json:
Auto-detected Project Settings (Vite):
- Build Command: vite build
- Development Command: vite --port $PORT
- Install Command: `yarn install`, `pnpm install`, `npm install`, or `bun install`
- Output Directory: dist
? Want to modify these settings? no
🔗 Linked to yongmi-changs-projects/yongmi-diary-final (created .vercel and added it to .gitignore)
🔍 Inspect: https://vercel.com/yongmi-changs-projects/yongmi-diary-final/4buX7ebaNsGysYfMN7KAXjVX2cJ8 [3s]
✅ Production: https://yongmi-diary-final-4uy897c69-yongmi-changs-projects.vercel.app [3s]
📁 Deployed to production. Run `vercel --prod` to overwrite later (https://vercel.link/2F).
💡 To change the domain or build command, go to https://vercel.com/yongmi-changs-projects/yongmi-diary-final/settings
PS C:\Workspaces\reactWork\yongmi_diary_final>
```

다시 배포할때는,,,,,
> vercel --prod